Grupo 12

GAMMA LAB 2.0

Software Design Document

(SDD)

Juan Felipe Romero Ortiz
Laura Juliana Garzón Arias
Sebastián Almanza Galvis
Xiomara Herrera Suárez

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
BOGOTÁ, D.C.
2026

GAMMA LAB 2.0

Autores:

Juan Felipe Romero Ortiz
Laura Juliana Garzón Arias
Sebastián Almanza Galvis
Xiomara Herrera Suárez

MEMORIA DE PROYECTO DE GRADO REALIZADO PARA CUMPLIR UNO DE LOS REQUISITOS PARA EL TÍTULO EN INGENIERÍA DE SISTEMAS

Director

Ing. Cécile Gauthier Umaña

Jurados del Trabajo Final de Grado

Ing. Nombre del Primer Jurado
Ing. Nombre del Segundo Jurado

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA

PROGRAMA DE INGENIERÍA DE SISTEMAS

BOGOTÁ, D.C.

Noviembre, 2026

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS

Rector de la Pontificia Universidad Javeriana

Nombre del Rector, S.J.

Decano de la Facultad de Ingeniería

Ing. Nombre del Decano de la Facultad

Director del Departamento de Ingeniería de Sistemas

Ing. Nombre del Director de Departamento, Ph.D.

Directora de Carrera de Ingeniería de Sistemas

Ing. Nombre de la Directora de Carrera

Artículo 23 de la Resolución No. 1 de Junio de 1946

"La Universidad no se hace responsable de los conceptos emitidos por sus alumnos en sus proyectos de grado. Sólo velará porque no se publique nada contrario al dogma y la moral católica y porque no contengan ataques o polémicas puramente personales. Antes bien, que se vean en ellos el anhelo de buscar la verdad y la Justicia"

Índice

1. INTRODUCCIÓN **1**
1.1. Propósito del documento . . . 1
1.2. Alcance . . . 1
1.3. Audiencia . . . 2

2. ARQUITECTURA **3**
2.1. Vista Lógica . . . 3
2.2. Vista Física . . . 5
2.2.1. Relaciones . . . 7
2.2.2. Versiones de las librerías . . . 7
2.3. Vista de Procesos . . . 8
2.4. Arquitectura de implementación . . . 9
2.4.1. Organización del código y reglas de dependencia . . . 10
2.4.2. Núcleo y ciclo de vida . . . 12
2.4.3. Arranque y cierre . . . 15
2.4.4. Hilos, tareas y recursos . . . 16
2.4.5. Fallos, registro y mensajes . . . 18
2.4.6. Extensibilidad . . . 20

3. DISEÑO DETALLADO DE COMPONENTES **23**
3.1. Modelo de datos y almacenamiento . . . 23
3.1.1. *TrialDataset* . . . 23
3.1.2. *SignalDataset* . . . 24
3.1.3. *DataStore* . . . 25
3.1.4. Archivo de proyecto *.glab* . . . 26
3.1.5. Diagrama de clases del modelo de datos . . . 27
3.2. Servicios de soporte . . . 28
3.2.1. *SettingsService* . . . 28
3.2.2. *FileIOService* . . . 29
3.2.3. *MeasurementService* . . . 30
3.2.4. *ExportService* . . . 31
3.2.5. *ProjectService* . . . 32
3.2.6. Diagrama de clases de los servicios de soporte . . . 33
3.3. Presentación . . . 34

3.3.1. Integración Qt/VTK 34
3.3.2. Retroalimentación al usuario 35
3.3.3. *VTKContextMenu* 36
3.3.4. *MainWindow* 37
3.3.5. Diagrama de clases de la presentación 38

4. COMPORTAMIENTO DEL SISTEMA 40
4.1. Diagrama abrir proyecto 40
4.2. Diagrama guardar proyecto 42
4.3. Diagrama cerrar proyecto 44
4.4. Diagrama *wavelet* 46

5. INTERFAZ DE USUARIO 48
5.1. Organización de la pantalla principal 48
5.2. Carga y guardado de archivos (pestaña *File*) 51
5.3. Preprocesamiento 53
5.4. *Analysis* 55

6. TRAZABILIDAD Y GESTIÓN DEL DISEÑO 61
6.1. Requisitos cubiertos 61
6.2. Casos de uso cubiertos 71

ÍNDICE DE FIGURAS

1. Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de *plugins*. 3
2. Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida. 6
3. Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema. 8
4. Diagrama de clases del modelo de datos y almacenamiento 28
5. Diagrama de clases de los servicios de soporte 33
6. Diagrama de clases de los elementos de presentación 39
7. Diagrama de secuencia para abrir proyecto 40
8. Diagrama de secuencia para guardar proyecto 42
9. Diagrama de secuencia para cerrar proyecto 44
10. Diagrama de secuencia para *wavelet* 46
11. Organización de la pantalla principal de Gamma Lab 48
12. opciones de *slope* 49
13. opciones de eliminar medida 49
14. opciones de *zoom* 50
15. explorador de archivos desplegado 50
16. resultados desplegados 50
17. Exportación de los resultados en formato CSV o XLSX 51
18. Pestaña *File* de Gamma Lab 51
19. Flujo de carga de la señal (*Open signal*) 52
20. Flujo de carga de proyecto (*Open project*) 52
21. Pestaña *Preprocessing* de Gamma Lab 53
22. opciones de *trials* 53
23. opciones de *remove artifact* 54
24. opciones de *filter* 54
25. opciones de FFT 55
26. opciones de FFT *Average* 56
27. opciones de PSD 56
28. opciones de *PSD Average* 57
29. opciones de *Average* 57
30. opciones de *Event Related Potential* 58
31. opciones de *Wavelet* 58
32. opciones de *Wavelet Average* 59
33. opciones de PAC 59

ÍNDICE DE TABLAS

1. Descripción de componentes de la Vista Lógica 5
2. Paquetes de primer nivel y componentes que alojan 10
3. Correspondencia entre ADR, principio y sección del SDD 12
4. Claves del registro de servicios del *Kernel* 13
5. Eventos publicados en el canal del *Kernel* 14
6. Severidades y tratamiento 19
7. Contrato *IPlugin* 20
8. Claves opcionales del descriptor *properties.yml* 21
9. Campos de *TrialDataset* 24
10. Estructuras de *trials* en *SignalDataset* 25
11. Claves usadas en el *DataStore* 26
12. Campos del manifiesto *.glab* 27
13. Preferencias guardadas por *SettingsService* 29
14. Interpretación de los formatos de archivo 30
15. Tipos de medición 30
16. Exportación de tablas 31
17. Pasos de las operaciones de proyecto 33
18. Utilidades de integración Qt/VTK 34
19. Presentación de los mensajes por severidad 35
20. Contenido del menú contextual de las gráficas 36
21. Responsabilidades de *MainWindow* 37
22. Requisitos funcionales cubiertos 66
23. Requisitos no funcionales cubiertos 71
24. Casos de uso cubiertos 76

1 INTRODUCCIÓN

1.1 Propósito del documento

El presente ***Software Design Document*** **(SDD)** tiene como propósito documentar el diseño técnico de GammaLab v2.0, describiendo la organización de sus componentes, las responsabilidades de sus módulos, las relaciones entre ellos y los mecanismos que permiten su interacción. El documento profundiza en las decisiones de diseño que orientan la implementación del sistema, complementando la descripción arquitectónica establecida en el ***Software Architecture Document*** **(SAD)** y los requisitos definidos en el ***Software Requirements Specification*** **(SRS)**.

Asimismo, el SDD proporciona una referencia técnica para comprender la estructura interna del *software*, analizar sus flujos de ejecución y orientar el desarrollo, mantenimiento y validación de sus funcionalidades.

1.2 Alcance

El alcance de este documento comprende la descripción de la arquitectura y el diseño detallado de GammaLab v2.0, desde la organización general de sus componentes hasta los elementos técnicos que intervienen en su funcionamiento. Para ello, incluye los siguientes aspectos:

- **Arquitectura del sistema:** Descripción de las vistas de escenarios, lógica, de despliegue, de procesos y física, así como de los principales componentes y sus responsabilidades.
- **Estructura del sistema:** Documentación de las clases, atributos, métodos y responsabilidades asociadas a los componentes principales.
- **Comportamiento del sistema:** Descripción de los flujos de ejecución y de las interacciones entre los componentes durante las operaciones principales.
- **Persistencia y modelos de datos:** Identificación de las estructuras utilizadas para representar y almacenar las señales, los ensayos *(trials)* y otros datos relevantes del sistema.
- **Relaciones entre componentes:** Descripción de las dependencias, los mecanismos de comunicación y la utilización de los servicios compartidos.

- **Interfaz de usuario:** Descripción de la organización de la interfaz, sus elementos de navegación y las principales funcionalidades disponibles para la gestión, el preprocesamiento y el análisis de señales.

1.3 Audiencia

La audiencia principal de este documento incluye:

- **Desarrolladores de *software*:** Proporciona información sobre la estructura interna del sistema, las responsabilidades de las clases, las dependencias entre componentes y los criterios de diseño que orientan la implementación y el mantenimiento.
- **Equipo de QA:** Sirve como referencia para contrastar el diseño y el comportamiento del sistema con los requisitos especificados, así como para identificar posibles inconsistencias entre lo definido y lo implementado.
- **Directora de tesis:** Permite evaluar la coherencia técnica del diseño, su correspondencia con los requisitos establecidos y el avance de las funcionalidades contempladas para GammaLab v2.0.
- ***Stakeholders*:** Ofrece una descripción técnica general de la solución, sus principales componentes y las funcionalidades que contempla, facilitando la comprensión de su organización y evolución.

2 ARQUITECTURA

Esta sección consolida la arquitectura de GammaLab 2.0 desde la perspectiva del *Software Design Document* (SDD), alineada con el *Software Architecture Document* (SAD). Su propósito es ofrecer una visión clara de los componentes, relaciones, flujos y decisiones que soportan el diseño e implementación descritos posteriormente en el Diseño Detallado.

A continuación, se desarrollan las vistas específicas que articulan la arquitectura hacia el diseño e implementación del sistema.

2.1 Vista Lógica

La vista lógica describe los componentes más importantes del sistema sin detallar a profundidad su instalación en hardware. Su objetivo es proporcionar una representación clara de cómo se organizan las responsabilidades y cómo interactúan los módulos fundamentales de GammaLab 2.0.

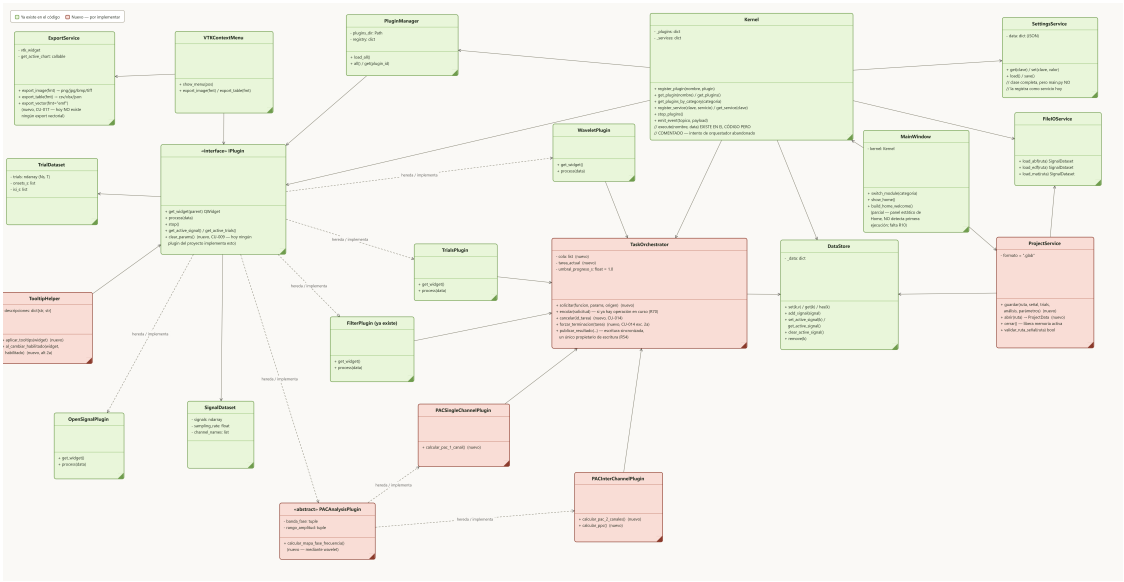


Figura 1: Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de *plugins*.

La Figura 1 ilustra los principales componentes del *software* y sus interfaces. La Tabla 1 describe cada uno de ellos.

Componente	Descripción	Relaciones / Dependencias
Kernel (núcleo)	Orquestador central de la aplicación: administra el registro de *plugins* y servicios, coordina la ejecución y media la comunicación entre módulos.	Provee *register_service()*/*get_service()* y *register_plugin()*; es consumido por *MainWindow* y por todos los *plugins*.
PluginManager / *loader*	Descubre e instancia los *plugins* disponibles a partir de sus carpetas y archivos *properties.yml*.	Utiliza *IPlugin* como contrato; registra cada instancia en el *Kernel*.
IPlugin (interfaz)	Contrato que toda extensión del sistema implementa (*initialize*, *start*, *stop*, *process*, *get_widget*).	Es implementada por todos los *plugins*; el *Kernel* la consume de forma polimórfica.
DataStore	Almacén centralizado en memoria de señales, *trials*, resultados intermedios y metadatos.	Registrado en el *Kernel*; lo consumen los *plugins*, *MainWindow*, *ProjectService*, *ExportService* y *MeasurementService* mediante claves canónicas.
FileIOService	Lee archivos de señal en formato ABF, EDF y MAT, y los convierte a la estructura interna *SignalDataset*.	Utiliza las librerías *EEGReaders* (*pyabf*, *pyedflib*) y, para MAT, SciPy y h5py; no escribe en el *DataStore*: entrega el *SignalDataset* al componente que lo invoca, que es quien lo registra.
ExportService	Genera archivos de salida (imagen y tabla) a partir de las visualizaciones activas.	Utiliza *VTK* para capturar el *render*; es consumido por los *plugins* desde el menú contextual.
MeasurementService	Gestiona las mediciones interactivas de pendiente y amplitud sobre los gráficos.	Utiliza *VTK* para la interacción; persiste sus resultados en *DataStore*.
SettingsService	Administra la configuración general del sistema y las preferencias del usuario.	Persiste en *gamma_lab_settings.json*; consumido por el *Kernel* y los *plugins*.
ProjectService	Gestiona la persistencia del espacio de trabajo (señal activa, *trials* y análisis) en archivos de proyecto *.glab*.	Consume *DataStore* para serializar y restaurar el estado completo de la sesión.

Componente	Descripción	Relaciones / Dependencias
TaskOrchestrator	Centraliza la ejecución de cómputo pesado (PAC, *Wavelet*, PSD) fuera del hilo de interfaz, con soporte de cancelación.	Es consumido por los *plugins*; se implementa como *TaskService*, con un único hilo trabajador (*QThread*) y sin procesos auxiliares (Sección 2.4.4).
Plugins (extensible)	Punto de extensión del sistema, organizados por categoría (E/S, Preprocesamiento, Análisis, Medición).	Consumen los servicios del *Kernel*; dependen de *PyQt5* y *VTK* para su interfaz.
MainWindow (APP)	Ventana principal de la aplicación; gestiona la navegación entre categorías de *plugins* y aloja su interfaz.	Consume el *Kernel* para descubrir y mostrar los *plugins* registrados.
PyQt5 (*framework*)	*Framework* de interfaz gráfica que administra ventanas, *widgets* y eventos del usuario.	Utilizado por el *Kernel*, *MainWindow* y los *plugins* para construir la interfaz.
VTK (librería)	Librería de visualización científica 2D/3D.	Usada directamente por los *plugins* de análisis y por *ExportService*/ *MeasurementService*.
EEGReaders (librería)	Conjunto de librerías (*pyabf*, *pyedflib*) para leer formatos de señal biomédica.	Utilizado por *FileIOService*.
ScientificLibs (librería)	Conjunto de librerías científicas (NumPy, SciPy, PyWavelets) para el cómputo numérico.	Utilizado por los *plugins* de análisis y por *TaskOrchestrator*.

Tabla 1: Descripción de componentes de la Vista Lógica

El *Kernel* centraliza el registro y consumo de servicios mediante *register_service()*/*get_service()*; el *DataStore* soporta la comunicación entre módulos mediante claves canónicas. El código se organiza en paquetes correspondientes a estas mismas responsabilidades (*io*, *preprocessing*, *analysis*, *measure*), lo que mantiene la correspondencia entre el diseño lógico y la implementación.

2.2 Vista Física

La vista física representa cómo se despliega y ejecuta GammaLab 2.0. Presenta los componentes físicos más importantes del sistema, sobre los cuales se instalan los componentes de *software*.

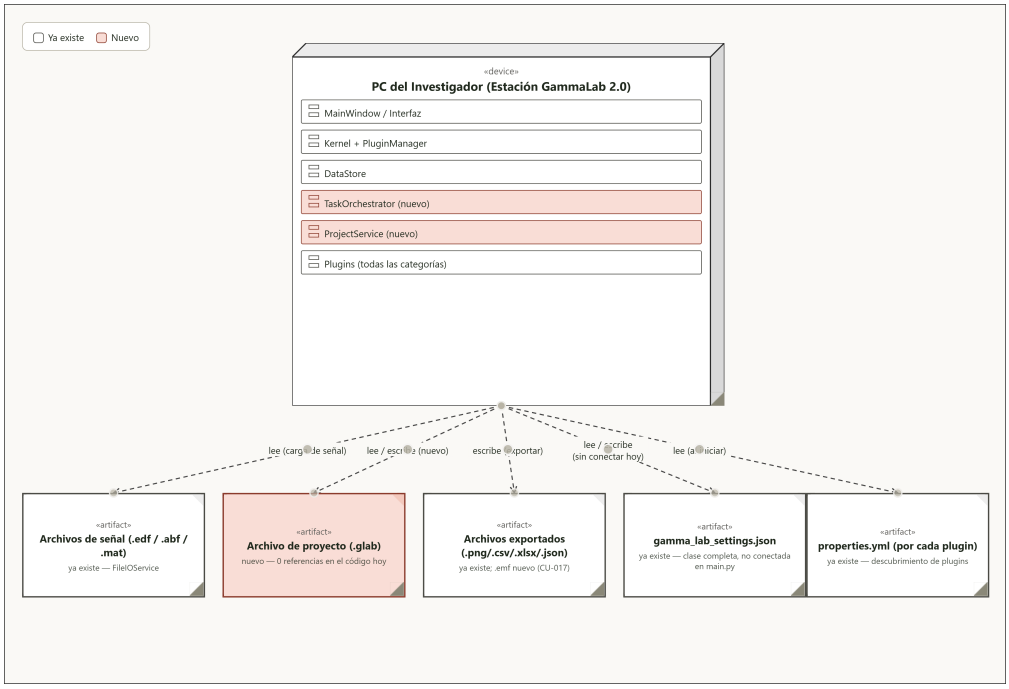


Figura 2: Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida.

La Figura 2 presenta la distribución física de los componentes del sistema en el entorno objetivo de ejecución. A continuación se describen los elementos principales:

- **«device» PC**: estación de trabajo del investigador donde se despliega y ejecuta el sistema completo; no existe arquitectura cliente-servidor.
- **«OS» Windows 10/11**: sistema operativo base; versión mínima Windows 10 de 64 bits.
- ***«executionEnvironment»* Python Runtime 3.11**: entorno de ejecución que sostiene la aplicación y sus librerías; versión mínima 3.11.x.
- **«artifact» *GammaLab.exe***: ejecutable principal, generado con PyInstaller, que inicia la aplicación y coordina la carga de todos los componentes.
- **«artifact» *Plugins* (**.py*)**: archivos de *plugin* que el *PluginManager* descubre e importa dinámicamente al arrancar, y que luego se registran en el *Kernel*.
- **«artifact» Archivos de señal** (*.edf*/*.abf*/*.mat*): leídos por *FileIOService*.
- **«artifact» Archivo de proyecto** (*.glab*): generado y leído por *ProjectService*.
- **«artifact» *gamma_lab_settings.json***: persistencia de configuración gestionada por *SettingsService*.

- **«artifact» *properties.yml*** (por cada *plugin*): sostiene el descubrimiento automático de *plugins*.

2.2.1 Relaciones

- La relación *loads* indica que el ejecutable principal carga y ejecuta los archivos de *plugin* durante el tiempo de ejecución.
- La relación *uses* especifica que el ejecutable principal utiliza los servicios y librerías desplegados dentro del mismo entorno de ejecución.

2.2.2 Versiones de las librerías

Las versiones ancladas para las librerías utilizadas, tal como se fijan en *requirements.txt*, son las siguientes:

- **PyQt5** = Versión 5.15.11 (*PyQt5-Qt5* 5.15.2)
- ***ScientificLibs***
  - NumPy = Versión 2.3.4
  - Pandas = Versión 2.3.3
  - SciPy = Versión 1.16.2
  - PyWavelets = Versión 1.9.0
- **VTK** = Versión 9.5.2
- **EEGReaders**
  - *pyABF* = Versión 2.3.8
  - *pyEDFlib* = Versión 0.1.42
- **Otras**
  - h5py = Versión 3.15.1
  - *PyYAML* = Versión 6.0.3
  - *PyQtWebEngine* = Versión 5.15.7

2.3 Vista de Procesos

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondición entre los casos de uso del sistema (p. ej. generar *trials* exige una señal previamente cargada).

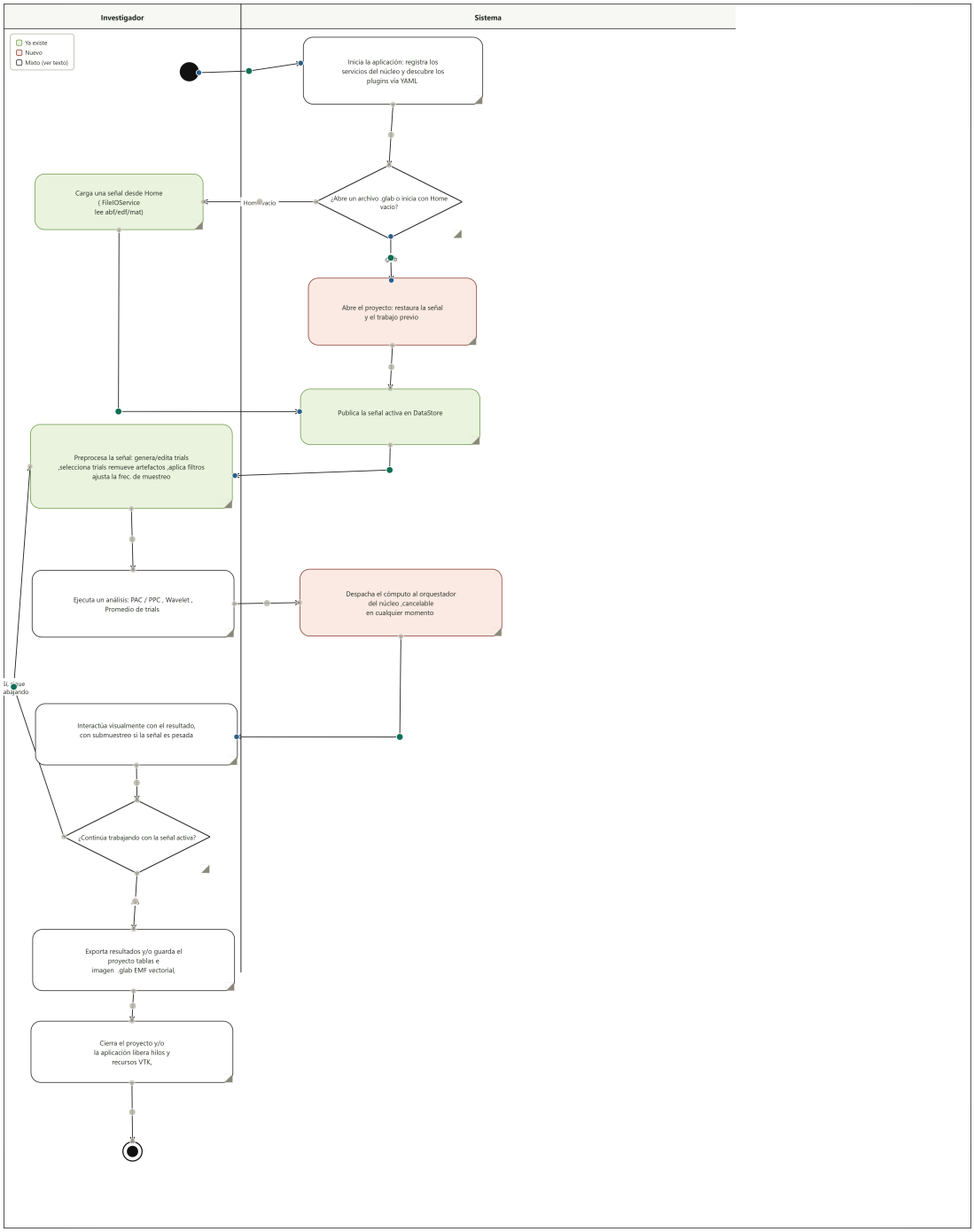


Figura 3: Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema.

El flujo de la Figura 3 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo, incluido el *TaskOrchestrator*, y descubre los *plugins* disponibles a partir de sus archivos *properties.yml*.
2. **Apertura o carga.** El investigador abre un archivo de proyecto *.glab* o carga una señal desde la pantalla *Home*; la señal activa se publica en *DataStore*.
3. **Preprocesamiento.** Generación y edición de *trials*, selección de *trials*, remoción de artefactos, filtrado y ajuste de la frecuencia de muestreo.
4. **Análisis.** Cálculo de acoplamiento fase-amplitud (PAC) de un canal o cruzado entre canales, transformada *Wavelet* y promedio de *trials*.
5. **Orquestación.** Todo cómputo pesado se despacha al *TaskOrchestrator* del núcleo, que lo ejecuta fuera del hilo de interfaz y permite su cancelación en cualquier momento.
6. **Visualización.** Interacción visual con el resultado, con submuestreo automático cuando la señal es de gran tamaño.
7. **Iteración o cierre.** El investigador retorna al preprocesamiento para continuar el análisis, o exporta y guarda el proyecto mediante *ProjectService* antes de cerrar la sesión.

La navegación entre secciones, la limpieza de formularios y la ayuda contextual mediante *TooltipHelper* son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en vez de forzarse dentro del flujo principal.

2.4 Arquitectura de implementación

Las tres vistas anteriores describen el sistema a nivel de componentes. Esta sección baja al nivel en que se escribe el código: qué paquete puede importar a cuál, quién gobierna el ciclo de vida de las extensiones, en qué orden arranca y se cierra la aplicación, cómo se reparte el trabajo entre hilos, cómo se tratan los fallos y qué hay que hacer para añadir una extensión nueva.

Cada apartado se cierra con el estado real de implementación. El *Software Architecture Document* (SAD) prescribe trece decisiones de arquitectura (ADR-01 a ADR-13); varias están implementadas, otras lo están en parte y otras todavía no. Documentar como existente lo que no se ha construido convertiría este documento en una descripción de un sistema imaginario, así que la distinción se mantiene explícita en cada caso.

2.4.1 Organización del código y reglas de dependencia

El código se organiza en cuatro paquetes de primer nivel. La Tabla 2 indica qué componente de la Vista Lógica (Sección 2.1) vive en cada uno.

Paquete	Contenido	Componentes del SAD
core/	Núcleo, contrato de extensión, servicios compartidos, modelos de datos y algoritmos que cruzan fronteras de extensión.	*Kernel*, *PluginManager*, IPlugin, *DataStore*, *FileIOService*, *ExportService*, *MeasurementService*, *SettingsService*, *ProjectService*, *TaskService*.
app/	Capa de presentación: ventana principal, pantalla de bienvenida y *widgets* propios.	*MainWindow*.
plugins/	Las extensiones, agrupadas por categoría (*io*, *preprocessing*, *analysis*, *measure*, *faq*).	*Plugins*.
test/	Pruebas automatizadas, con la misma división por responsabilidad (*services_test*, *plugins_test*, *ui_test*, *performance_test*).	N/A

Tabla 2: Paquetes de primer nivel y componentes que alojan

Dentro de *core/* la división es por responsabilidad: *core/kernel.py* (el núcleo), *core/plugins/* (contrato, descubrimiento y registro), *core/services/* (los servicios compartidos), *core/model/* (las estructuras de datos), *core/filters/* (los algoritmos compartidos) y *core/utils/* (utilidades transversales).

Reglas de dependencia. Son tres, y se enuncian en orden de importancia.

1. ***core/* no importa de *app/* ni de *plugins/*.** Es la regla que sostiene el estilo microkernel: el núcleo tiene que poder existir sin ninguna extensión concreta y sin la ventana principal. Si se invierte, el núcleo deja de ser un núcleo y pasa a ser una capa más del monolito.

2. **Las extensiones no se importan entre sí.** Lo que dos extensiones comparten sube a *core/*; lo que usa una sola se queda en su carpeta. Es la regla que hizo que *gabor.py*

pasara del *plugin* de PAC a *core/filters/* cuando PPC empezó a necesitarlo, y la que mantiene a la *wavelet* de Morse dentro de PAC, porque solo PAC la usa.

3. **El cómputo no importa de Qt ni de VTK.** El módulo *compute.py* de cada extensión recibe arreglos de NumPy y devuelve arreglos de NumPy. Es lo que permite ejecutarlo en un hilo trabajador y probarlo sin levantar una interfaz gráfica (ADR-04).

Cómo se verifica. La primera regla se comprueba automáticamente: basta recorrer los archivos de *core/* y buscar importaciones de *app* o *plugins*. Al momento de redactar esta sección el resultado es **cero importaciones**, frente a 3 de *app/* hacia los otros paquetes y 32 de *plugins/*, que es la dirección correcta. La tercera regla se comprueba de la misma forma sobre cada *compute.py*, y está cubierta por pruebas: *test/services_test/test_gabor_en_core.py* verifica con el módulo *ast* que *core/filters/gabor.py* no importe nada de *plugins*, mirando las sentencias de importación y no el texto, porque los comentarios sí nombran a *plugins* y eso es documentación, no acoplamiento.

Correspondencia con las decisiones del SAD. La Tabla 3 relaciona cada decisión de arquitectura con el apartado de este documento que la desarrolla y con su estado de implementación.

ADR	Princ.	Decisión	Sección	Estado
ADR-01	P1	Estilo microkernel con núcleo propio	2.4.1, 2.4.2	Implementada
ADR-02	P1	Ciclo de vida como máquina de estados gobernada por el *Kernel*	2.4.2	**No implementada**
ADR-03	P2	Catálogo de eventos con carga útil tipada	2.4.2	**No implementada**
ADR-04	P3	Cómputo en un módulo sin dependencias de Qt ni VTK	2.4.1, 2.4.6	Parcial (9 de 19 extensiones)
ADR-05	P4	Descubrimiento declarativo validado	2.4.2, 2.4.6	Parcial
ADR-06	P5	*DataStore* con escritor único	2.4.4	Parcial
ADR-07	P6	Estrategia de memoria	2.4.4	Parcial
ADR-08	P7	Liberación determinista de recursos	2.4.3, 2.4.4	Parcial
ADR-09	P8	*TaskOrchestrator* híbrido (hilos y procesos)	2.4.4	Parcial (solo hilos, por medición)
ADR-10	P9	Control de admisión por estimación de costo	2.4.4	**No implementada**
ADR-11	P10	Manejador global de excepciones	2.4.5	**No implementada**

ADR	Princ.	Decisión	Sección	Estado
ADR-12	P11	Registro estructurado con archivo rotativo	2.4.5	**No implementada**
ADR-13	N/A	Python, PyQt5 y VTK como base	2.2	Implementada

Tabla 3: Correspondencia entre ADR, principio y sección del SDD

2.4.2 Núcleo y ciclo de vida

Kernel. Es un *QObject* con cuatro responsabilidades y ninguna más: mantener el diccionario de extensiones registradas, mantener el diccionario de servicios, exponer un canal de eventos y recorrer las extensiones para detenerlas. No calcula, no dibuja y no lee archivos. Su interfaz pública es:

- *register_plugin(name, plugin)*: registra la extensión y le llama *initialize(kernel)*;
- *get_plugin(name)*, *get_plugins()*, *get_plugins_by_category(cat)*: consulta;
- *register_service(key, service)* / *get_service(key)*: el localizador de servicios;
- *emit_event(topic, payload)*: publicación de eventos;
- *stop_plugins()*: detención ordenada.

PluginManager. Recorre el árbol de *plugins/* buscando archivos *properties.yml*, instancia la clase declarada en cada uno y guarda el par *(meta, instancia)* en un registro indexado por el identificador de la extensión. El descubrimiento es tolerante a fallos: una carpeta cuyo descriptor esté mal formado, o cuya clase no pueda importarse, se omite con un mensaje en consola y el resto de la aplicación sigue arrancando.

Estados y transiciones. Hoy el ciclo de vida de una extensión es una secuencia, no una máquina de estados. La extensión pasa de *descubierta* (el *PluginManager* la instancia) a *inicializada* (el *Kernel* le llama *initialize*), luego a *activa* (*start* desde *MainWindow* al entrar en su pestaña, que pone el indicador *started*) y, por último, a *detenida* (*stop*).

> **Brecha respecto al SAD (ADR-02).** El SAD prescribe seis estados explícitos (descubierta, inicializada, activa, inactiva, detenida y **fallida**) con las transiciones gobernadas únicamente por el núcleo. La implementación actual no mantiene el estado de cada extensión: el único indicador es el booleano *started* de *IPlugin*, las

transiciones las dispara *MainWindow* y no *Kernel*, y no existe el estado *fallida*: una extensión cuya carga falla se omite en silencio y no aparece en la interfaz, en lugar de aparecer marcada como fallida. Cerrar esta brecha es un trabajo acotado (un campo de estado en el *Kernel* y una transición por método), pero no se ha hecho.

Registro de servicios. Los servicios se registran y se consultan por una clave de texto. Las ocho claves vigentes, junto con el servicio que identifican y el punto del código en que se registran, se recogen en la Tabla 4.

Clave	**Servicio**	**Dónde se registra**
"DataStore"	*DataStore*	*main.py*, al arrancar.
"FileIO"	*FileIOService*	*main.py*, al arrancar.
"TaskService"	*TaskService*	*main.py*, al arrancar.
"Settings"	*SettingsService*	*registrar_servicios_compartidos()*, en *core/services/compartidos.py*.
"Export"	*ExportService*	*registrar_servicios_compartidos()*.
"Measurement"	*MeasurementService*	*registrar_servicios_compartidos()*.
"MainWindow"	*MainWindow*	Constructor de *MainWindow*; *main.py* vuelve a registrarla una vez construida.
"ProjectService"	*ProjectService*	Constructor de *MainWindow*, que crea el servicio.

Tabla 4: Claves del registro de servicios del *Kernel*

Brecha: las claves no están centralizadas. Hoy cada clave se escribe como literal en el punto de uso, tanto al registrar como al consultar en cada extensión. Un error de escritura no se detecta: *get_service()* devuelve *None* y el fallo aparece más tarde y lejos de su causa. Lo que corresponde es un módulo con las claves como constantes, por ejemplo *core/services/claves.py*, y que tanto el registro como el consumo las importen de ahí. Nótese además que la nomenclatura no es uniforme: la clave del servicio de entrada/salida es *"FileIO"* y no *"FileIOService"*, mientras que la del proyecto sí coincide con el nombre de la clase (*"ProjectService"*); la discrepancia sobrevive precisamente porque no hay un único sitio donde estén escritas.

Catálogo de eventos. El canal es un único *pyqtSignal(str, object)*: un tópico de texto y una carga útil sin tipo. Los tópicos que efectivamente se emiten son cuatro, y se recogen en la Tabla 5.

Tópico	**Carga útil**	**Significado**
signal_added	*{"key": nombre}*	Se cargó una señal nueva. Lo emiten la extensión *Open Signal* y *ProjectService* al abrir un proyecto.
trials_generated	Variable según el emisor: *{"signal", "channel"}* en *Trials* y *Remove Artifact*, *{"key"}* en *Open Signal* y solo *{"channel"}* en *Filter*.	Se recortaron o se modificaron los *trials* de un canal.
current_trial_changed	Estado del selector de *trials*	Una extensión publicó el *trial* con el que trabaja, mediante *IPlugin.publish_current_trial()*.
trial_discard_updated	*{"signal", "channel", "index"}*	Se descartó o recuperó un *trial*.

Tabla 5: Eventos publicados en el canal del *Kernel*

> **Brecha respecto al SAD (ADR-03).** El SAD prescribe un catálogo con carga útil tipada, de modo que un error de tópico se detecte al emitir. Con el canal genérico actual, publicar *"signal_add"* en lugar de *"signal_added"* no produce ningún error: simplemente no lo escucha nadie. El código actual ya presenta dos casos de este tipo: *MainWindow*, *IPlugin* y las extensiones *Trials* y *PPC* se suscriben al tópico *signal_active_changed*, y *PPC* también a *trials_changed*, pero ningún componente emite ninguno de los dos, de modo que esas ramas nunca se ejecutan. A ello se suma que la carga útil de *trials_generated* no es uniforme entre emisores. La tabla anterior documenta el catálogo *de facto*; convertirlo en contrato exige constantes para los tópicos y una estructura declarada por carga útil.

Validaciones de carga. El descubridor exige que el *properties.yml* contenga las ocho claves obligatorias (*id*, *name*, *category*, *subcategory*, *version*, *icon*, *logic_class*, *ui_class*) y que la clase declarada en *logic_class* exista dentro de la carpeta. Lo que **no** valida, y el SAD prescribe en ADR-05:

- **Versión del contrato.** El campo *version* del descriptor es la versión de la extensión, no la del contrato *IPlugin* contra el que fue escrita. Una extensión compilada contra un *IPlugin* antiguo se carga igual y falla al usarse.
- **Dependencias.** No existe un campo para declararlas. Una extensión que necesite un servicio no registrado lo descubre en tiempo de ejecución, con un *None*.
- **Unicidad del identificador.** El registro es un diccionario indexado por *id*: dos extensiones con el mismo identificador se sobrescriben sin aviso. El *Kernel* sí avisa por consola de un *nombre* repetido, pero igualmente lo sobrescribe.

2.4.3 Arranque y cierre

CU-011: Iniciar la aplicación. El orden de instanciación en *main.py* no es arbitrario; cada paso depende del anterior:

1. **Guarda del temporizador de VTK.** Antes de crear ningún *widget* se sustituye el manejador de temporizador de *QVTKRenderWindowInteractor* por una versión que comprueba que el *widget* siga visible. Va primero porque la sustitución tiene que estar puesta antes de que exista el primer interactor.
2. ***QApplication***, fuentes, icono y hoja de estilos.
3. **Pantalla de bienvenida**, que informa del avance de los pasos siguientes.
4. ***Kernel***.
5. **Servicios del núcleo**: *DataStore*, *FileIO* y *TaskService*, seguidos de los servicios compartidos *Settings*, *Export* y *Measurement*, que *registrar_servicios_compartidos()* crea una sola vez (el *DataStore* debe existir antes, porque los dos últimos lo reciben al construirse). En este mismo paso se asigna el *Kernel* a *VTKContextMenu*, para que los menús contextuales de las gráficas obtengan de él los servicios compartidos. *Van antes del descubrimiento de extensiones*: *register_plugin()* llama a *initialize(kernel)*, y desde ahí una extensión ya puede pedir cualquier servicio. Si estas líneas se movieran después, recibiría *None*.
6. **Descubrimiento e instanciación** de las extensiones con *PluginManager.load_all()*.
7. **Registro en el *Kernel*** de cada extensión, por el nombre declarado en su descriptor.

8. ***MainWindow***, que se construye con el *Kernel* ya poblado (necesita la lista de extensiones para montar la barra de categorías) y se registra como servicio, porque las extensiones la consultan para la barra de estado y los diálogos. Durante su construcción crea y registra además el *ProjectService*.

9. **Mostrar la ventana** en el primer ciclo del bucle de eventos.

Inyección de servicios. No hay un contenedor de inyección de dependencias: el patrón es *Service Locator*. Una extensión no recibe sus dependencias en el constructor, sino que las pide al *Kernel* cuando las necesita, con *kernel.get_service(clave)*. El contrato *IPlugin* ofrece envoltorios para los accesos más frecuentes (*get_datastore()*, *get_active_signal()*, *get_active_trials()*, *get_trials_de_canal()*), que resuelven el servicio y además traducen la ausencia en un aviso al usuario en lugar de en una excepción.

CU-022: Cerrar la aplicación. El cierre recorre las extensiones registradas y llama al *stop()* de cada una. Qué libera ese *stop()* depende de la extensión; las de análisis que usan el orquestador cancelan además las tareas que tengan en vuelo con *cancel_all_from(id)*, de modo que no quede un hilo trabajador escribiendo sobre una interfaz que ya no existe.

> **Brecha respecto al SAD (ADR-08).** El SAD exige que tras el cierre «no queden hilos, procesos ni contextos gráficos activos». La detención actual no garantiza eso en un caso concreto: una tarea cancelada que no responda en dos segundos se descarta pero no se detiene, porque un hilo de Python no puede terminarse de forma segura desde fuera. El hilo desacoplado sigue vivo hasta que su función termine. Está documentado en el SAD como limitación conocida de ADR-09.

2.4.4 Hilos, tareas y recursos

Reglas de hilos para el desarrollador. Son tres y no admiten excepción:

1. **VTK y Qt solo en el hilo de interfaz.** Ninguna de las dos librerías es segura fuera de él. Un cálculo que toque un *vtkTable* desde un trabajador produce una violación de acceso, no una excepción.

2. **Ninguna extensión crea sus propios hilos.** El cómputo pesado se encola en el orquestador. Antes había extensiones con su propio *QThread*, lo que daba tantos patrones de concurrencia como extensiones y ninguna forma común de cancelar.

3. **El resultado se dibuja en el *slot* de *finished***, que Qt ejecuta ya en el hilo de interfaz, y siempre tras comprobar que el manejador recibido sea el vigente.

TaskService. Es la implementación del *TaskOrchestrator* del SAD. Mantiene una cola única y ejecuta **una tarea a la vez** en un hilo trabajador. *submit(fn, owner=...)* devuelve un *TaskHandle* con cuatro señales (*progress*, *finished*, *failed* y *cancelled*) y al cómputo se le inyecta un *TaskContext* con *ctx.cancelled* y *ctx.progress(pct, mensaje)*.

Las extensiones disponen de dos formas de encolar un cálculo. La recomendada es *IPlugin.run_task(fn, on_done, on_failed=..., on_cancelled=..., on_finished=...)*, que concentra el patrón en un único lugar: cancela la tarea que la extensión tenga todavía en curso, encola la nueva, conecta las cuatro señales con la guarda de vigencia y ejecuta *on_done* en el hilo de interfaz; cuando no hay *TaskService* disponible, como en las pruebas sin interfaz, ejecuta la función en línea con las mismas retrollamadas. Once extensiones la utilizan: las cinco de frecuencia, *Average*, ERP, *Trials*, *Filter*, *Open Signal* y PPC. La segunda forma es invocar directamente *TaskService.submit()* y conectar las señales en la propia extensión. Es la que emplean *Remove Artifact*, *Wavelet*, *Wavelet Average*, PAC y el índice de modulación, y la que combinan con *run_task() Open Signal* y PPC en los cálculos que requieren un control más fino del progreso o de varias tareas encadenadas.

En ambos casos cada señal se conecta con una guarda. Al cambiar un parámetro se encola otra tarea y la anterior puede seguir viva un instante; sin la guarda *if handle is not self._handle*, su resultado sobrescribiría al nuevo. El fragmento siguiente muestra la forma directa:

```
handle = tasks.submit(cp.ppc_mapa, owner=self.meta.id, ...)
self._handle = handle
handle.progress.connect(partial(self._on_progress, handle))
handle.finished.connect(partial(self._on_done, handle))
handle.failed.connect(partial(self._on_failed, handle))
handle.cancelled.connect(partial(self._on_cancelled, handle))
```

Control de admisión. Existe en una forma mínima: al encolar, el servicio descarta las tareas *pendientes* de la misma extensión (*_discard_queued_from(owner)*), de modo que pulsar *Generate* cinco veces no deja cinco cálculos en cola. No existe, en cambio, la estimación de costo previa que prescribe ADR-10: el sistema no se niega a lanzar un cálculo que vaya a tardar minutos ni advierte de ello.

Cancelación. Es cooperativa: se activa un *threading.Event* y el cómputo lo consulta en sus puntos de comprobación. Si no responde en dos segundos, el servicio deja de esperarlo y descarta su resultado, pero el hilo sigue corriendo.

Política hilo/proceso. El SAD prescribe en ADR-09 un orquestador híbrido con un proceso trabajador para los cómputos aislables. **La implementación usa solo hilos**, y la razón es una medición, no un olvido: el costo de arrancar un proceso y serializar los arreglos

hacia él resultó ser de **1.486 ms** frente a los **1.163 ms** que tarda el cálculo en sí. El proceso costaba más que lo que se quería paralelizar. Como las operaciones pesadas son transformadas de NumPy, que liberan el *GIL* mientras calculan, los hilos sí aprovechan varios núcleos: el reparto de *trials* entre cuatro hilos en PPC rinde entre **1,8**× y **2,7**× con resultado idéntico bit a bit. Lo que los hilos no dan es el aislamiento de fallos ni la cancelación forzada, que siguen siendo el argumento a favor de ADR-09 y quedan pendientes.

Política de memoria. Cuatro mecanismos, en orden de impacto:

- **Submuestreo.** Todos los análisis toman una frecuencia de muestreo objetivo y submuestrean antes de calcular. El factor es entero, así que la tasa pedida no siempre se alcanza: *tasa_efectiva()* devuelve la tasa *real* resultante, y es esa, y no la pedida, la que construye el eje de frecuencias. Tratarlas como iguales corría todo el eje hasta un 11 %.
- **Tipo numérico por etapa.** El cálculo va en *float64* y lo que se entrega a VTK para dibujar se convierte a *float32*, que basta para la pantalla y ocupa la mitad.
- **Cómputo por tandas.** Las transformadas tiempo-frecuencia se calculan en bloques de frecuencias en lugar de reservar la matriz completa: para 500 filas por 60.000 muestras la diferencia es de 480 MB a 30 MB.
- **Memoización de filtros.** El diseño de los filtros IIR está memoizado y devuelve una copia de los coeficientes, para que ningún consumidor pueda alterar la entrada compartida. Al cambiar de *trial*, el ahorro medido es de **7,25**×.

Liberación determinista. Al salir de una pestaña, *stop()* desactiva el interactor de VTK de la extensión. Está documentado como defecto abierto que la memoria no se libera por completo al cerrar un proyecto.

2.4.5 Fallos, registro y mensajes

Taxonomía de errores. El sistema distingue tres severidades, cada una con su canal, a través de *PluginAlerts*:

Severidad	Canal	Cuándo se usa
Error	Diálogo modal (*alerts.error*)	El usuario pidió algo que no se puede hacer y tiene que enterarse: un archivo ilegible, un parámetro fuera de rango, dos canales no alineados. Interrumpe.
Aviso	Diálogo o registro (*alerts.warning*)	Se puede continuar, pero hay algo que condiciona la lectura del resultado.
Informativo	Registro y barra de estado	Avance de una tarea, resultado de una operación, condiciones del cálculo. No interrumpe.

Tabla 6: Severidades y tratamiento

Los avisos sobre el resultado de un cálculo no abren diálogo, y hay una razón técnica además de la de usabilidad: un modal abre un bucle de eventos anidado, y el temporizador de *QVTKRenderWindowInteractor* sigue disparando dentro de él sobre *widgets* que pueden estar ocultos. Tres cierres inesperados de la aplicación se rastrearon hasta ahí. Por eso las extensiones que informan de condiciones del cálculo (el índice de modulación y PPC) lo hacen con una función que devuelve una lista de textos, que el *plugin* registra. Un fallo de verdad sí abre diálogo, pero ese es otro camino: el *slot* de *failed*.

Captura en los trabajadores. Una excepción dentro de un cómputo no cruza el límite del hilo: el *TaskService* la captura y la convierte en la señal *failed* con su mensaje, que el *slot* correspondiente traduce en un diálogo y en la restauración del botón. El dibujo también va protegido: una excepción al renderizar se registra y se reporta, y el bloque *finally* restituye el estado de la interfaz pase lo que pase.

Archivos corruptos. *FileIOService* valida al leer y lanza una excepción con el motivo; la extensión que la invoca la traduce en un diálogo. Una carpeta de extensión con el descriptor mal formado no impide el arranque: se omite con un mensaje y el resto carga.

Brecha respecto al SAD (ADR-11 y ADR-12). Dos elementos prescritos no existen en el código.

No hay manejador global de excepciones. No se instala ningún *sys.excepthook*. Una excepción no capturada en el hilo de interfaz, fuera de los bloques protegidos, termina el proceso sin que el usuario sepa por qué y sin que quede registro.

No hay registro estructurado. El sistema no usa el módulo *logging*: los mensajes salen por *print()*, a través de *IPlugin._log()*, que antepone el nombre de la extensión. No hay niveles, ni marca de tiempo, ni archivo, ni rotación. Diagnosticar un fallo que le ocurrió al usuario exige reproducirlo, que es exactamente lo que ADR-12 pretende evitar. La migración está acotada (un *logging.getLogger* por módulo y un *RotatingFileHandler* en el arranque), pero toca todas las llamadas existentes a *print*.

2.4.6 Extensibilidad

Contrato *IPlugin*. Toda extensión hereda de *IPlugin* y recibe su descriptor en el constructor. Los métodos que debe implementar son tres (*stop()*, *get_widget(parent)* y *process(data)*) y hereda *initialize(kernel)*, *start(kernel)*, el acceso a los servicios y el manejo de alertas. La Tabla 7 resume el contrato.

Método	Origen	Responsabilidad
initialize(kernel)	Heredado	Guardar la referencia al *Kernel*. Se llama al registrar.
start(kernel)	Heredado	Obtener *MainWindow*, suscribirse al canal de eventos y marcar *started*.
stop()	**Obligatorio**	Liberar lo propio: tareas en vuelo, interactor de VTK, observadores.
get_widget(parent)	**Obligatorio**	Construir la vista la primera vez y devolverla siempre.
process(data)	**Obligatorio**	Recibir datos del *Kernel* o de otra extensión.
get_analysis_params() / *apply_analysis_params(d)*	Opcional	Guardar y restaurar los parámetros con el proyecto.

Tabla 7: Contrato *IPlugin*

Descriptor *properties.yml*. Ocho claves obligatorias: *id* (identificador único), *name* (el que se muestra y con el que se registra), *category* y *subcategory* (dónde aparece el botón; la interfaz agrupa por subcategoría sin lista fija, de modo que una subcategoría nueva aparece sola), *version*, *icon* (archivo dentro de la carpeta), *logic_class* (la clase que implementa *IPlugin*) y *ui_class*. Las claves opcionales, definidas en *PluginMeta* (*core/plugins/meta.py*),

se recogen en la Tabla 8. Como el descriptor se convierte directamente en una instancia de *PluginMeta*, una clave no prevista en la clase impide cargar la extensión, que se omite con un mensaje en consola.

Clave	**Valor por defecto**	**Uso**
description	Vacía	Texto de la ayuda emergente del botón.
saves_analysis	*false*	Indica que los parámetros de la extensión se guardan en el archivo de proyecto (Sección 3.1.4).
order	Sin valor	Posición del botón dentro de su subcategoría.
subcategory_order	Sin valor	Posición de la subcategoría; solo se aplica en la sección *Analysis*.
free_graphics	*true*	Permite que *MainWindow* libere la ventana OpenGL del módulo cuando deja de estar en pantalla.

Tabla 8: Claves opcionales del descriptor *properties.yml*

Patrón *compute / ui*. Una extensión de análisis se escribe en tres archivos:

- *compute.py*: el cálculo. Recibe NumPy, devuelve NumPy. No importa Qt ni VTK. Es lo que corre en el trabajador y lo que se prueba.
- *<nombre>_plugin_ui.py*: la vista: los controles y el hueco del gráfico.
- *<nombre>_plugin.py*: la lógica de coordinación: lee los parámetros, encola, recibe y dibuja.

La separación no es estética. Permite probar el cálculo sin interfaz (las 76 pruebas del cómputo de PPC corren en segundos y sin Qt) y permite ejecutarlo en un hilo trabajador, que es lo que exige la regla de hilos. Nueve de las diecinueve extensiones siguen hoy este patrón; las de frecuencia (*fft*, *psd* y sus promedios) todavía tienen el cálculo dentro de la clase del *plugin*.

Puntos de extensión. Hay tres, de distinta madurez:

1. **Extensiones.** Es el punto principal y el único verdaderamente declarativo: basta una carpeta con *properties.yml* bajo *plugins/* para que el sistema la descubra al arrancar. No hay que tocar ningún registro central.

2. **Formatos de archivo y de exportación.** No son declarativos. Un formato de lectura nuevo exige un método en *FileIOService* y una rama en el punto de llamada; uno de exportación, una rama en *ExportService*. Convertirlos en puntos de extensión con registro (un diccionario de extensión de archivo a función lectora) está pendiente.

3. **Eventos.** Cualquier extensión puede publicar un tópico nuevo y cualquiera puede escucharlo, pero sin catálogo ni tipado (véase 2.4.2).

Guía para crear una extensión nueva. Los pasos, en orden:

1. Crear la carpeta bajo la categoría que corresponda, por ejemplo *plugins/analysis/time_frequency/<nombre>/*.

2. Escribir *properties.yml* con las ocho claves y colocar el icono.

3. Escribir *compute.py* con el cálculo puro, y sus pruebas en *test/plugins_test/*. Conviene hacerlo antes que la interfaz: es donde están los errores que importan.

4. Escribir *<nombre>_plugin_ui.py* con los controles y el área del gráfico.

5. Escribir *<nombre>_plugin.py*: heredar de *IPlugin*, implementar *get_widget*, *stop* y *process*, y encolar el cálculo con *run_task()*, que ya conecta las cuatro señales con su guarda. Solo si se necesita un control más fino se recurre a *submit()* y a la conexión manual descrita en la Sección 2.4.4.

6. Si el cálculo es compartido con otra extensión, subirlo a *core/filters/* en lugar de duplicarlo.

7. Verificar que la regla de dependencia se mantiene: que *compute.py* no importe Qt ni VTK, y que nada de *core/* importe la extensión nueva.

No hay que modificar *main.py*, ni el *Kernel*, ni *MainWindow*: el descubrimiento es automático y la agrupación por categoría también.

3 DISEÑO DETALLADO DE COMPONENTES

Esta sección describe las clases que forman GammaLab 2.0: los datos que guardan, las operaciones que ofrecen, las reglas que cumplen y los componentes con los que colaboran. El *Kernel*, el *PluginManager*, el contrato *IPlugin*, el *TaskService* y *PluginAlerts* se describieron en la Sección 2.4 y aquí solo se referencian.

Los componentes se presentan de abajo hacia arriba, de modo que cada uno se describe después de aquellos en los que se apoya. Primero va el modelo de datos, luego los servicios de soporte que operan sobre él, después los elementos de presentación que muestran la información al usuario y, por último, una vista de conjunto con el diagrama de clases completo.

3.1 Modelo de datos y almacenamiento

El modelo de datos sigue el recorrido de la información desde que se abre un archivo hasta que se guarda el trabajo: el archivo de señal se convierte en un *SignalDataset*; de este se recortan los *TrialDataset*; las señales cargadas se almacenan en el *DataStore* y, finalmente, ese estado se guarda en el archivo *.glab*.

Un archivo de registro (*.abf*, *.edf* o *.mat*) se convierte en una señal multicanal. De un canal de esa señal se recortan *trials*. Las señales cargadas forman el estado de la sesión, y ese estado se guarda en disco como un proyecto. Ninguna de estas clases depende de Qt ni de VTK.

3.1.1 *TrialDataset*

En un experimento electrofisiológico se aplican estímulos repetidos y se registra la respuesta. Un *trial* es el fragmento del registro de un canal que rodea a uno de esos estímulos. Empieza t_0 segundos respecto al instante del estímulo, llamado *onset*, y termina t_1 segundos después. Como todos los *trials* se recortan con la misma ventana, tienen el mismo número de muestras y se pueden comparar o promediar muestra a muestra.

TrialDataset, definido en *core/model/trial_dataset.py*, es un *dataclass*: una clase de Python cuyo propósito es agrupar datos con tipos declarados. Una instancia no representa un *trial* aislado sino todos los *trials* de un canal, guardados en una matriz de tamaño (N_s, T). N_s es el número de muestras de cada *trial* y T el número de *trials*, de modo que cada columna de la matriz es un *trial*. Por ejemplo, con una frecuencia de muestreo de 10 kHz, una ventana de $-0{,}1$ a 0,5 s y 40 estímulos detectados, la matriz tiene 6000 filas y 40 columnas. Todas las columnas comparten el mismo eje de tiempo relativo. La Tabla 9 resume los campos de la clase.

Campo	Contenido
source, *channel_index*, *channel_name*, *unit*	Archivo de origen, canal recortado y su unidad.
sampling_rate	Frecuencia de muestreo (Hz).
t0, *t1*, *time_rel* (N_s)	Ventana del recorte y eje de tiempo relativo.
trials (N_s, T)	Muestras en *float64*; una columna por *trial*.
onsets_s (T), *isi_s*	Instante de cada estímulo e intervalo entre estímulos.
metadata	Información sobre cómo se obtuvo el conjunto.

Tabla 9: Campos de *TrialDataset*

En los metadatos se guardan, bajo la clave *generation_params*, los parámetros exactos del recorte: canal, canal de estímulo, umbral, ventana y número de estímulos esperados. Con esos parámetros se pueden volver a recortar los mismos *trials* a partir de la señal, y en esa posibilidad se basa el archivo de proyecto (Sección 3.1.4).

Los *trials* se generan con la función de recorte de *core/filters/trials.py*, que se ejecuta como tarea del *TaskService*. Cuando la ventana de un *trial* se sale del registro, las muestras faltantes se rellenan con ceros para que la matriz conserve su forma rectangular. Si no se detecta ningún estímulo, el resultado es un único *trial* con el canal completo. Al construirse, la clase verifica que la matriz sea bidimensional y que el eje de tiempo y la lista de *onsets* tengan longitudes coherentes con ella; si alguna condición falla, la construcción se interrumpe con un error. *TrialDataset* no tiene métodos propios: las operaciones sobre los *trials* pertenecen a *SignalDataset*.

3.1.2 *SignalDataset*

SignalDataset, definido en *core/model/signal_dataset.py*, cumple dos funciones: representar la señal multicanal tal como se leyó del archivo y gestionar los conjuntos de *trials* recortados de ella. Sus instancias las crea *FileIOService* y el resto del sistema accede a ellas a través del *DataStore*.

La señal se guarda en una matriz de tamaño (C, N), con una fila por canal, acompañada del eje de tiempo y del nombre y la unidad de cada canal. El nombre del archivo, con su extensión, sirve como identificador de la señal en todo el sistema. Si la frecuencia de muestreo que trae el encabezado del archivo es incorrecta, el usuario puede corregirla: la clase recalcula el eje de tiempo, registra que la frecuencia fue modificada a mano y conserva el valor original para poder restaurarlo.

Los *trials* se guardan en estructuras privadas, que solo se modifican con los métodos de la clase. Todas se indexan por el par (archivo, canal), como muestra la Tabla 10.

Estructura	**Contenido**
Conjuntos originales	Los *TrialDataset* recortados, en orden de generación.
Descartes	Por (archivo, canal), los índices de los *trials* excluidos.
Conjuntos derivados	Por (archivo, canal), el resultado del preprocesamiento.
Caché	Por (archivo, canal), el último resultado de la consulta de *trials* activos.

Tabla 10: Estructuras de *trials* en *SignalDataset*

Cada canal tiene un solo conjunto de *trials*: al recortar de nuevo un canal, el conjunto nuevo reemplaza al anterior y los de los demás canales se conservan. El canal del último conjunto recortado es el canal activo. Los análisis de un canal lo usan por defecto; los de dos canales, como PAC intercanal y PPC, piden cada canal por su nombre.

El usuario puede descartar *trials* y recuperarlos después. Descartar no borra datos: el *trial* queda marcado y se excluye al momento de consultar, como si se aplicara una máscara sobre la matriz. Los índices se refieren siempre a la numeración original del recorte.

El preprocesamiento (filtrado y modificación de artefactos) no escribe sobre los *trials* recortados. Trabaja sobre una copia, el conjunto derivado, que la clase acepta solo si corresponde a un conjunto original existente, tiene su misma forma y no comparte con él los datos. Desde ese momento los análisis leen el derivado, y el original sigue disponible.

Todas estas reglas confluyen en *get_active_trials*, la consulta que usan los análisis para obtener sus datos. Toma el conjunto del canal indicado, o el del canal activo si no se indica ninguno. Si existe un derivado, lo usa en su lugar. Luego excluye los *trials* descartados y guarda el resultado en la caché, donde se reutiliza mientras los descartes no cambien. Cualquier operación que modifica los *trials* elimina la información que dependía del estado anterior: recortar de nuevo un canal borra sus descartes, su derivado y su caché, y descartar un *trial* o registrar un derivado invalida la caché de ese canal.

3.1.3 *DataStore*

El *DataStore*, definido en *core/services/data_store.py*, es un almacén clave–valor en memoria que contiene el estado compartido de la sesión, empezando por las señales. Existe una única instancia, creada al arrancar la aplicación y registrada en el *Kernel*, desde donde la obtienen los demás componentes.

Como salvaguarda, la extensión *Open Signal* crea y registra un *DataStore* o un *FileIO-Service* nuevos si no encuentran el servicio en el *Kernel*; en el arranque normal esa rama no se ejecuta, porque ambos servicios se registran antes del descubrimiento de extensiones.

El almacén admite un solo escritor. Al crearse guarda el identificador del hilo que lo creó, que es el hilo de interfaz, y rechaza con un error cualquier escritura que provenga de otro hilo. Por esta razón, una tarea que corre en un hilo trabajador no escribe directamente en el *DataStore*: entrega su resultado al terminar, y la escritura se realiza en el hilo de interfaz. Lecturas y escrituras toman además un candado, de modo que ningún hilo lee el almacén mientras se está modificando. Este control cubre las claves del almacén, no el interior de los objetos guardados: una consulta devuelve una referencia al objeto, por lo que una señal obtenida del almacén se modifica directamente con sus propios métodos.

El almacén no impone un esquema y acepta cualquier clave. Las que usa el sistema se resumen en la Tabla 11. Los resultados numéricos de los análisis no se guardan en el *DataStore*; permanecen en la extensión que los calculó.

Clave	**Valor**
Nombre del archivo (p. ej. *23n09000.abf*)	La señal (*SignalDataset*).
active_signal	Clave de la señal activa.
measurements	Lista de mediciones registradas por *MeasurementService*.
current_trial	*Trial* en que se trabaja, mostrado en la barra de título.
_project_service	Referencia al *ProjectService* para componentes sin acceso al *Kernel*.

Tabla 11: Claves usadas en el *DataStore*

Cuando el usuario carga una señal nueva o cierra el proyecto, el almacén libera las señales y, con ellas, sus *trials*, las mediciones y el *trial* en curso, para que la memoria quede disponible antes de cargar la siguiente señal.

3.1.4 Archivo de proyecto *.glab*

El archivo *.glab* es la forma en disco del estado descrito en los apartados anteriores; lo escribe y lo lee *ProjectService*. Un proyecto es una carpeta que contiene el manifiesto

<nombre>.glab en formato JSON, una copia del archivo de señal y la subcarpeta *archivos/*, destino por defecto de las exportaciones. Como la señal está copiada en la carpeta, el proyecto puede abrirse en otro equipo.

El manifiesto no contiene matrices de datos. Guarda la información necesaria para volver a producirlas a partir del archivo de señal: los *trials* se registran por sus parámetros de recorte, sus descartes y las recetas de preprocesamiento aplicadas, y los análisis por los parámetros configurados en cada extensión. Las mediciones son la excepción y se guardan completas, porque son pocos valores y dependen de puntos marcados a mano por el usuario. La Tabla 12 describe los campos.

Campo	**Contenido**
schema_version	Versión del formato (*"1.0"*).
project_name, *saved_at*	Nombre del proyecto y fecha del guardado.
signal	Nombre del archivo, formato, frecuencia de muestreo y su origen.
trials	Por canal: *generation_params*, índices descartados y recetas de preprocesamiento.
measurements	Mediciones completas: puntos marcados, contexto y valores.
analysis	Parámetros de cada extensión que declara *saves_analysis: true*.

Tabla 12: Campos del manifiesto *.glab*

Al abrir un proyecto, el estado se reconstruye en el mismo orden en que se produjeron los datos. Primero se lee la señal y, si su frecuencia de muestreo había sido corregida, se aplica el valor guardado. Después se recortan de nuevo los *trials*, se reaplican los descartes y, sobre un conjunto derivado, las recetas de preprocesamiento. Por último se restauran las mediciones y se entregan sus parámetros a cada extensión. Los descartes se aplican antes que las recetas porque cada receta registra los descartes vigentes en el momento en que se aplicó.

3.1.5 Diagrama de clases del modelo de datos

La Figura 4 muestra las clases descritas en esta sección con sus atributos y operaciones. Las líneas indican que el *DataStore* almacena las señales, que cada *SignalDataset* contiene los conjuntos de *trials* originales y derivados de sus canales, y que el *DataStore* lanza *EscrituraNoPermitida* ante una escritura desde otro hilo.



Figura 4: Diagrama de clases del modelo de datos y almacenamiento

3.2 Servicios de soporte

Los servicios de soporte son los objetos que operan sobre el modelo de datos: leen archivos y los convierten en señales, registran mediciones, exportan resultados y guardan o abren proyectos. Se presentan de menor a mayor dependencia: el primero no depende de ningún otro componente y el último reúne a casi todos los anteriores.

3.2.1 *SettingsService*

SettingsService, definido en *core/services/settings_service.py*, guarda las preferencias de la aplicación que deben conservarse entre sesiones. Las mantiene en un diccionario en memoria y las escribe en el archivo *gamma_lab_settings.json*, en la carpeta de la aplicación. Al crearse lee el archivo; si no existe o no se puede interpretar, empieza con un diccionario vacío. Cada vez que se modifica un valor, el archivo se escribe de inmediato.

El servicio se crea una sola vez, en el registro de servicios compartidos del núcleo, y se registra en el *Kernel* con la clave *"Settings"*. *MainWindow*, el menú contextual de las gráficas y los *plugins* lo piden al *Kernel*; ninguno crea el suyo. Las preferencias que se guardan hoy son las carpetas usadas por última vez, que el sistema propone como punto de partida en los diálogos de archivo, y un indicador que evita repetir el panel de bienvenida en cada arranque (Tabla 13).

Clave	Contenido
last_open_dir	Última carpeta desde la que se cargó una señal.
last_project_dir	Última carpeta desde la que se abrió un proyecto.
last_export_dir	Última carpeta de exportación fuera de un proyecto.
bienvenida_mostrada	Indica que el panel de bienvenida ya se mostró, de modo que no vuelva a abrirse automáticamente al arrancar.

Tabla 13: Preferencias guardadas por *SettingsService*

3.2.2 *FileIOService*

FileIOService, definido en *core/services/fileio_service.py*, convierte un archivo de registro en un *SignalDataset* (Sección 3.1.2). Tiene un método de lectura por formato, y los tres entregan la señal con la misma estructura, de modo que el resto del sistema no depende del formato de origen. Se registra en el *Kernel* al arrancar la aplicación.

El servicio no escribe en el *DataStore* ni usa la interfaz gráfica, por lo que puede ejecutarse en un hilo trabajador. Lo invocan el *plugin* de carga de señales, como tarea del *TaskService*, y *ProjectService* al abrir un proyecto. Si el archivo no puede leerse o su contenido es inconsistente, el servicio lanza una excepción con el motivo, y el componente que lo invocó la presenta al usuario. La Tabla 14 resume cómo se interpreta cada formato.

Formato	Biblioteca	Interpretación
ABF	pyabf	Si el archivo tiene varios barridos (*sweeps*), se toma el primero de cada canal. Todos los canales deben tener el mismo número de muestras. La frecuencia de muestreo, los nombres y las unidades se leen del encabezado.
EDF	pyedflib	Primero se lee el encabezado de cada canal. Si todos comparten frecuencia y longitud, se leen directamente; si no, se remuestrean por interpolación lineal a la frecuencia más baja y a la duración más corta. Los canales sin nombre o sin unidad reciben valores por defecto.
MAT (clásico)	scipy	Se toma la primera variable numérica de una o dos dimensiones y se orienta con una fila por canal. Como el formato no incluye la frecuencia de muestreo, se asigna 1 Hz y se marca como valor por defecto para que el usuario la corrija.

Formato	Biblioteca	Interpretación
MAT (v7.3)	h5py	Cada variable numérica se interpreta como un canal; los de menor longitud se completan con valores vacíos (*NaN*). La frecuencia se trata igual que en el formato clásico.

Tabla 14: Interpretación de los formatos de archivo

3.2.3 *MeasurementService*

MeasurementService, definido en *core/services/measurement_service.py*, permite al usuario medir sobre una gráfica marcando dos puntos con el ratón. Hay una sola instancia del servicio, registrada en el *Kernel* con la clave *"Measurement"*, que guarda las mediciones en el *DataStore* (Sección 3.1.3). Como cada gráfica tiene su medición en curso y sus líneas dibujadas, el servicio entrega a cada menú contextual una sesión de medición propia, atada a la ventana de esa gráfica; la sesión recibe la forma de obtener la gráfica activa. Trabaja en el hilo de interfaz, porque opera directamente sobre los objetos de VTK.

Una medición avanza por tres estados: en reposo, esperando el primer punto y esperando el segundo. El usuario elige el tipo de medición, marca los dos puntos y el servicio vuelve al reposo; la tecla Esc cancela en cualquier momento. Cada clic se ajusta al punto de datos más cercano dentro de un radio de 10 píxeles, de modo que la medición usa valores reales de la serie y no la posición exacta del cursor. Un clic que se desplaza más de unos pocos píxeles se interpreta como arrastre y se ignora. Mientras se espera el segundo punto, el *zoom* y el desplazamiento quedan suspendidos para que los ejes no cambien entre un punto y otro. Los cálculos numéricos se realizan con las funciones de *core/filters/measurements.py*. La Tabla 15 resume los tipos de medición.

Tipo	Resultado
Pendiente	Pendiente entre los dos puntos, sus diferencias en X y en Y, y la distancia entre ellos.
Amplitud	Amplitud pico a pico dentro de la ventana definida por los dos puntos, con el máximo, el mínimo y sus posiciones.
Pendiente en todos los *trials*	Una medición de pendiente por cada *trial*, en las mismas posiciones; solo disponible en el *plugin Trials*.

Tabla 15: Tipos de medición

Cada medición se guarda en el *DataStore* con un identificador correlativo por tipo (*slope-001*, *amplitude-002*, . . .), los dos puntos, la fecha y su contexto: vista, gráfica, *trial*, canal, *plugin* y dominio (tiempo o frecuencia). Sobre la gráfica, la medición se dibuja como una línea discontinua entre los dos puntos. El contexto determina dónde se vuelve a dibujar: al cambiar de *trial* o de vista, el servicio borra los trazos visibles y redibuja solo las mediciones cuyo contexto coincide con el actual. El usuario puede ocultar o mostrar los trazos y eliminar la última medición, una en particular o todas. Cada cambio en la lista se notifica para que el panel de resultados se actualice.

3.2.4 *ExportService*

ExportService, definido en *core/services/export_service.py*, escribe en archivos externos el contenido de una gráfica o los resultados de un análisis. Al igual que *MeasurementService*, hay una sola instancia, registrada en el *Kernel* con la clave *"Export"*, que entrega a cada menú contextual un exportador atado a la ventana de su gráfica. El servicio guarda lo que comparten todas, la carpeta de exportación, que sale de los ajustes y del proyecto abierto; cada exportador recibe la forma de obtener la gráfica activa y los nombres de la señal, el canal y el *plugin*. Con esos nombres arma el nombre propuesto para el archivo, en la forma *señal_canal_plugin_fecha.ext*.

La exportación de imágenes captura la ventana de la gráfica y la escribe como PNG, JPG, BMP o TIFF. La exportación vectorial genera un archivo EMF en el que ejes, marcas, etiquetas y barra de color son elementos vectoriales, y los mapas de calor se incrustan como imagen. Para las tablas, el sistema tiene tres caminos, que resume la Tabla 16.

Qué se exporta	**Origen**	**Formatos**
Datos de una gráfica	Menú contextual de la gráfica. Se exportan todas las series con sus datos completos, no la versión reducida que se dibuja en pantalla.	CSV, XLSX, JSON
Resultados con varias tablas (*Wavelet*, PAC, índice de modulación, PPC)	El *plugin* arma las tablas (parámetros, resumen, matrices) y el servicio las escribe: en CSV una tras otra, en XLSX una hoja por tabla y en JSON una lista por tabla.	CSV, XLSX, JSON
Tablas de mediciones (pendiente y amplitud)	Panel de resultados. A diferencia de los dos casos anteriores, las escriben directamente los *plugins* de mediciones, sin pasar por *ExportService*.	CSV, XLSX

Tabla 16: Exportación de tablas

Los archivos XLSX se generan con un escritor propio (*core/utils/xlsx_writer.py*), sin bibliotecas externas. La carpeta propuesta depende de si hay un proyecto abierto: con proyecto, es su subcarpeta *archivos/* (Sección 3.1.4); sin proyecto, es la última carpeta de exportación guardada en *SettingsService*. Las exportaciones hechas dentro de un proyecto no modifican esa preferencia.

3.2.5 *ProjectService*

ProjectService, definido en *core/services/project_service.py*, guarda y abre los proyectos descritos en la Sección 3.1.4. Es el servicio con más dependencias: usa *FileIOService* para leer la señal, el *DataStore* para registrar y consultar el estado, el *TaskService* para las operaciones lentas, el algoritmo de recorte para reconstruir los *trials* y las extensiones para obtener y restaurar sus parámetros. Lo crea *MainWindow*, que lo registra en el *Kernel* y además lo guarda en el *DataStore*, para que el menú contextual de las gráficas pueda consultar la carpeta del proyecto.

El servicio mantiene el estado del proyecto abierto: su carpeta, la ruta del manifiesto, su nombre y un indicador de cambios sin guardar. El nombre por defecto se forma con el nombre del archivo de señal seguido de *_gammalab*. Las extensiones activan el indicador cuando el usuario modifica algo que forma parte del proyecto, y *MainWindow* lo consulta para pedir confirmación antes de descartar cambios. Cada vez que el proyecto se abre, se guarda, cambia de nombre o se cierra, el servicio avisa a *MainWindow* para que actualice la barra de título.

Guardar y abrir se dividen en pasos, porque una parte requiere la interfaz y otra puede tardar. La Tabla 17 muestra el reparto.

Operación	**Paso**	**Hilo**
Guardar	1. Decidir el destino. La primera vez se pide una carpeta; se rechazan la raíz del disco y las carpetas del sistema (Escritorio, Documentos, Descargas, Imágenes, Música, Videos y la carpeta del usuario).	Interfaz
	2. Copiar el archivo de señal a la carpeta, en bloques de 8 MB, con progreso y posibilidad de cancelar.	Trabajador
	3. Escribir el manifiesto con el estado actual y los parámetros de las extensiones.	Interfaz
Abrir	1. Leer el manifiesto y el archivo de señal, sin tocar el *DataStore*.	Trabajador
	2. Registrar la señal y reconstruir el estado en el orden descrito en la Sección 3.1.4.	Interfaz
Cerrar	Restablecer el estado del proyecto.	Interfaz

Operación	Paso	Hilo

Tabla 17: Pasos de las operaciones de proyecto

La copia de la señal se escribe primero en un archivo temporal, que se renombra al terminar; así, una copia cancelada o fallida nunca deja una señal incompleta en el proyecto. Si el archivo ya está en la carpeta con el mismo tamaño y la misma fecha de modificación, no se copia de nuevo. Si el guardado se cancela o falla, el proyecto conserva la ubicación que tenía antes, y si el proyecto cambió de nombre desde el último guardado, el manifiesto se renombra al guardar. Cerrar el proyecto solo restablece el estado del servicio; la liberación de las señales y de la memoria la realiza *MainWindow* a través del *DataStore* (Sección 3.1.3).

3.2.6 Diagrama de clases de los servicios de soporte

La Figura 5 muestra los cinco servicios con sus atributos y operaciones. Las cajas con la anotación «ver» corresponden a componentes descritos en otras secciones y aparecen solo para mostrar con qué se relaciona cada servicio; las líneas indican qué crea, lee o escribe cada uno.

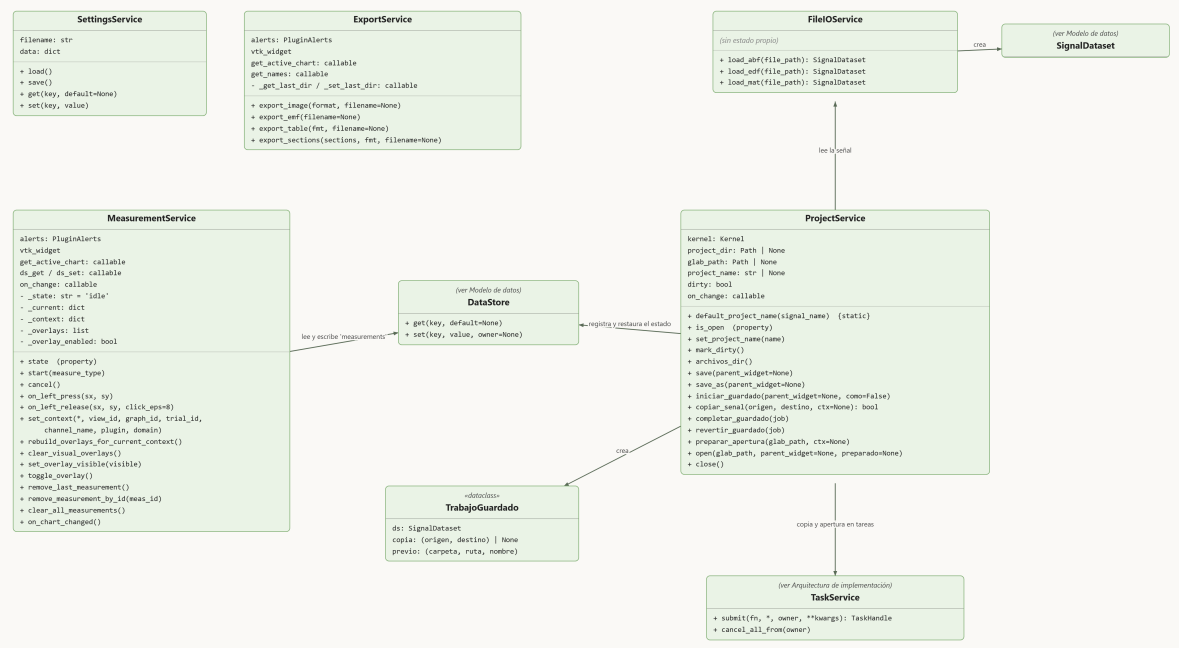


Figura 5: Diagrama de clases de los servicios de soporte

3.3 Presentación

Los elementos de presentación muestran la información al usuario y reciben sus acciones. Se presentan de las piezas al contenedor: primero las utilidades que integran VTK con Qt, que no dependen de otros componentes; después los elementos que informan al usuario del estado del sistema y el menú contextual de las gráficas, que reúne los servicios de soporte; y por último la ventana principal, que aloja a todos los anteriores y a las extensiones. Todos estos elementos trabajan en el hilo de interfaz.

3.3.1 Integración Qt/VTK

Las gráficas de GammaLab se dibujan con VTK dentro de *widgets* de Qt. Cada extensión crea su propio *widget* VTK, y la ventana principal conserva esos *widgets* al cambiar de módulo para no perder el trabajo del usuario. Esta combinación se apoya en un conjunto de utilidades, ubicadas en *core/utils/*, que resuelven la conversión de datos, el dibujo de series largas, la interacción con el ratón y el manejo de los recursos de las gráficas que no están en pantalla. La Tabla 18 las resume.

Módulo	**Función**
vtk_temporizador	Agrega una comprobación al temporizador interno del *widget* VTK de Qt, para que no actúe sobre un *widget* oculto o destruido.
adapters	Convierte una señal o una matriz de *trials* en la tabla de columnas que VTK usa para dibujar.
submuestreo	Reduce las series largas a los puntos que se dibujan en pantalla.
chart_coords, *cursor_readout*	Convierten píxeles de pantalla en coordenadas de los ejes y muestran la posición del cursor.
vtk_observers	Registra los observadores de eventos de cada gráfica para poder retirarlos y reponerlos.
vtk_ventanas	Libera la ventana OpenGL de un módulo que deja de estar en pantalla.

Tabla 18: Utilidades de integración Qt/VTK

El *widget* VTK de Qt incluye un temporizador que se ejecuta cada 10 ms sin comprobar el estado del *widget*. La utilidad del temporizador reemplaza, una sola vez al arrancar la aplicación, el método que atiende ese temporizador por una versión que no actúa si el *widget*

no está visible, no tiene ventana o ya fue destruido. Como el reemplazo se hace sobre la clase, cubre los *widgets* de todas las extensiones, incluidas las que se agreguen en el futuro.

El adaptador de datos construye una tabla con una columna de tiempo y una columna por canal o por *trial*; la de la señal se guarda en el propio *SignalDataset* (Sección 3.1.2) y se reutiliza mientras la señal no cambie. El submuestreo limita cada serie a unos 2000 puntos dibujados: divide la serie en tramos y conserva el mínimo y el máximo de cada uno, de modo que los picos siguen visibles. Al acercar la gráfica, la serie se reduce de nuevo sobre el rango visible y recupera el detalle hasta la resolución original. Solo se reduce lo que se dibuja: los datos completos se conservan, y son los que leen la exportación y las mediciones (Sección 3.2).

La lectura del cursor muestra la posición del ratón en unidades de los ejes, en una esquina de la gráfica y en la barra de estado. Cuando la gráfica dibuja curvas, el valor mostrado corresponde al punto de la señal más cercano al cursor, tomado de la curva que pasa más cerca; en los mapas de calor y en las demás gráficas sin curvas es la posición libre del cursor. Los observadores de eventos (movimiento del ratón, clics) se registran a través de una utilidad que los recuerda por *widget*, de modo que la ventana principal pueda retirarlos cuando el usuario sale de un módulo y reponerlos cuando vuelve. Por último, la ventana OpenGL de un módulo oculto puede liberarse: se descartan su contexto gráfico y su memoria, pero se conservan el *widget*, la escena y los resultados, y VTK vuelve a crear la ventana la próxima vez que el módulo se dibuja.

3.3.2 Retroalimentación al usuario

Los mensajes al usuario se emiten a través de *PluginAlerts* (Sección 2.4) con tres severidades, cada una con su propio canal de presentación, como muestra la Tabla 19.

Severidad	**Presentación**
Información	Tarjeta azul no modal que se cierra sola a los pocos segundos.
Advertencia	Tarjeta ámbar no modal que permanece hasta que el usuario la cierra.
Error	Diálogo modal, porque interrumpe la operación en curso.

Tabla 19: Presentación de los mensajes por severidad

Las tarjetas no modales las administra *NotificationCenter* (*core/utils/notifications.py*), que las apila en la esquina inferior izquierda del área de trabajo, con un máximo de cuatro visibles, y no repite un mismo aviso si se emite varias veces seguidas. Cada tarjeta es una ventana nativa, porque las gráficas de VTK también lo son y cubrirían a un *widget* común.

El avance de las tareas lo muestra *TaskProgressBar* (*core/utils/task_progress.py*), una barra única ubicada en la barra de estado de la ventana principal. Está conectada al *TaskService* y aparece automáticamente cada vez que una extensión encola un cálculo, con el porcentaje, un mensaje y un botón para cancelar, sin que la extensión tenga que hacer nada. La barra de estado muestra además mensajes breves coloreados según su severidad y la lectura del cursor descrita en la Sección 3.3.1.

3.3.3 *VTKContextMenu*

VTKContextMenu, definido en *core/utils/vtk_context_menu.py*, es el menú que aparece al hacer clic derecho sobre una gráfica. Cada gráfica tiene su propia instancia, creada por la extensión al dibujar. Al construirse recibe la gráfica, su *widget* y los nombres de la señal, el canal y la extensión. No crea servicios propios: obtiene del *Kernel* las instancias únicas de *ExportService* y *MeasurementService* (claves *"Export"* y *"Measurement"*) y les pide, respectivamente, un exportador y una sesión de medición atados a la ventana de esa gráfica, mediante *para()* y *sesion_para()*. Su función es reunir en un solo lugar las acciones comunes a todas las gráficas y delegarlas en el servicio que corresponde. La Tabla 20 muestra el contenido del menú.

Submenú	**Acciones**	**Delega en**
Zoom	*Zoom* horizontal, vertical o en ambos ejes; restablecer la vista.	El propio menú
Measurements	Pendiente, amplitud, pendiente en todos los *trials*; cancelar; mostrar u ocultar trazos; eliminar la última medición o todas.	*MeasurementService*
Export as image	PNG, JPG, BMP, TIFF y EMF vectorial.	*ExportService*
Export table	CSV, JSON y XLSX.	*ExportService*

Tabla 20: Contenido del menú contextual de las gráficas

Además del menú, el componente configura el *zoom* con la rueda del ratón (sobre los dos ejes, o sobre uno solo si se mantiene pulsada la tecla Ctrl o Shift), conecta la lectura del cursor y reenvía los clics de la gráfica al servicio de mediciones. Las extensiones pueden agregar acciones propias al menú. También proporciona al servicio de exportación la carpeta de destino descrita en la Sección 3.2.4, que consulta en el *ProjectService* guardado en el *DataStore*.

3.3.4 *MainWindow*

MainWindow, definida en *app/view/main_window.py*, es la ventana principal de la aplicación y el contenedor de todos los elementos anteriores y de las extensiones. Se construye después de registrar las extensiones, porque necesita conocerlas para armar la navegación, y se registra en el *Kernel* como servicio. Su estructura visual se define en *Ui_MainWindow* (*main_window_ui.py*), una clase generada a partir del diseño de la interfaz y sin lógica propia. Al construirse, la ventana crea el *ProjectService*, el centro de notificaciones y la barra de progreso de tareas. Sus responsabilidades se agrupan como muestra la Tabla 21.

Grupo	**Responsabilidad**
Navegación	Secciones *Home*, *Preprocessing* y *Analysis*; los botones de cada sección se agrupan por subcategoría y se ordenan según el *properties.yml* de cada extensión. Incluye una barra de búsqueda con autocompletado sobre los nombres de las extensiones.
Área de trabajo	Muestra la vista de la extensión seleccionada y administra el cambio entre módulos.
Proyecto	Menú Archivo: cargar señal, guardar, guardar como, abrir y cerrar proyecto, coordinando *ProjectService* y *TaskService*.
Barra de título	Nombre de la señal activa, nombre del proyecto (editable con doble clic) e indicador del *trial* en uso.
Barra de estado	Mensajes por severidad, barra de progreso y lectura del cursor.
Panel de resultados	Tablas de mediciones de pendiente y amplitud, con exportación a CSV y XLSX.
Explorador	Vista de la carpeta del proyecto abierto, con operaciones básicas sobre archivos.
Cierre	Confirmación de cambios sin guardar y cierre ordenado de la aplicación.

Tabla 21: Responsabilidades de *MainWindow*

El cambio entre módulos sigue siempre la misma secuencia. Al salir de un módulo, la ventana cancela sus tareas en el *TaskService*, lo detiene, retira los observadores de sus gráficas y oculta su vista; a continuación, salvo que el módulo lo desactive en su *properties.yml*, libera su ventana OpenGL una vez mostrado el módulo siguiente. Al entrar en un módulo, la ventana lo inicia la primera vez, construye su vista o reutiliza la existente, repone sus observadores y la muestra. Las vistas se conservan durante toda la sesión, de modo que el usuario encuentra cada módulo como lo dejó.

La barra de título refleja el estado de la sesión. El nombre del proyecto usa *EditableLabel* (*app/view/widgets/editable_label.py*), una etiqueta que se convierte en campo de texto al hacer doble clic y comunica el nuevo nombre al *ProjectService*. El indicador de *trial* muestra con qué *trials* trabaja el módulo visible, numerados por su posición entre los válidos; se actualiza cada vez que una extensión publica un cambio y cambia de color cuando el *trial* mostrado está descartado. Cuando no hay ningún módulo activo, la sección *Home* muestra un panel de bienvenida, y el área de trabajo muestra el logotipo de la aplicación mientras no haya una señal cargada.

Al cargar una señal nueva o cerrar el proyecto, la ventana libera la señal anterior: cancela las tareas en curso, destruye las vistas de los módulos, pide a cada extensión que suelte sus referencias a la señal y libera las señales del *DataStore* (Sección 3.1.3). Al cerrar la aplicación, pide confirmación si hay cambios sin guardar, cancela todas las tareas, detiene las extensiones y finaliza las ventanas OpenGL de todos los módulos antes de salir. Durante el arranque, antes de que exista la ventana principal, *SplashScreen* (*app/view/splash_screen.py*) muestra una imagen con una barra de progreso y el paso en curso, según la secuencia descrita en la Sección 2.4.

3.3.5 Diagrama de clases de la presentación

La Figura 6 muestra los elementos de presentación con sus atributos y operaciones. *MainWindow* ocupa el centro porque crea o aloja a casi todos los demás; las cajas con la anotación «ver» remiten a componentes descritos en otras secciones.

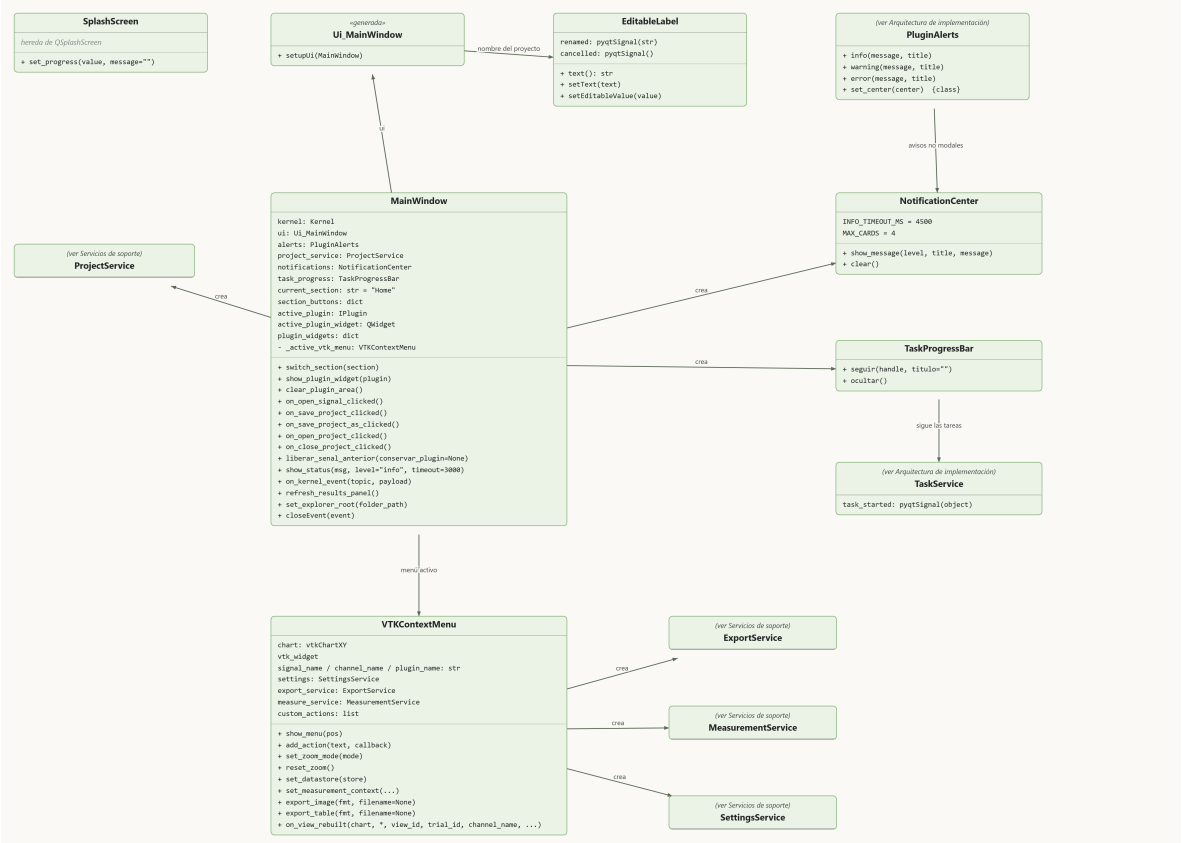


Figura 6: Diagrama de clases de los elementos de presentación

4 COMPORTAMIENTO DEL SISTEMA

4.1 Diagrama abrir proyecto

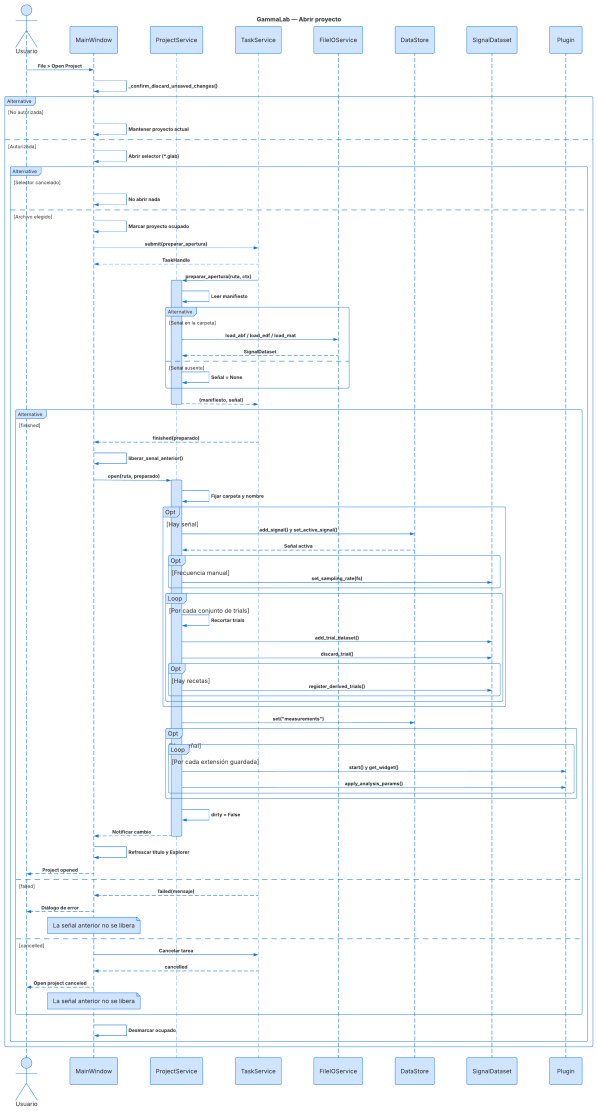


Figura 7: Diagrama de secuencia para abrir proyecto

Este diagrama representa el flujo que sigue GammaLab 2.0 cuando el usuario abre un proyecto desde la opción ***File > Open project***. El proceso comienza en *MainWindow*, que invoca *_confirm_discard_unsaved_changes()* para verificar si es posible abandonar el proyecto actual. Es el mismo mecanismo de confirmación descrito en el diagrama de cerrar proyecto: si hay un cálculo en curso o cambios sin guardar, se consulta al usuario, y si la

apertura no se autoriza, el proyecto actual se mantiene sin cambios.

Cuando la apertura se autoriza, *MainWindow* muestra el selector de archivos filtrado por la extensión *.glab*. Si el usuario lo cierra sin elegir un archivo, no se abre nada. Si elige uno, *MainWindow* marca el proyecto como ocupado y envía la lectura a *TaskService* mediante *submit(preparar_apertura)*. El servicio devuelve inmediatamente un *TaskHandle* y ejecuta el trabajo en un hilo trabajador, de modo que la interfaz permanece disponible y el usuario puede seguir el avance en la barra de progreso.

Dentro de la tarea, *ProjectService* ejecuta *preparar_apertura()*, que lee el manifiesto del proyecto y busca el archivo de señal en la carpeta. Si lo encuentra, lo carga mediante *FileIOService* con el método que corresponde a su formato (*load_abf*, *load_edf* o *load_mat*) y obtiene un *SignalDataset*; si no lo encuentra, la señal queda vacía. En esta etapa no se modifica el *DataStore* ni la interfaz: la tarea solo devuelve el manifiesto y la señal leída.

Al terminar, *TaskService* comunica el resultado a *MainWindow* mediante una de tres señales. Cuando la lectura finaliza correctamente (*finished*), *MainWindow* ejecuta *liberar_senal_anterior()*, el mismo procedimiento descrito en el diagrama de cerrar proyecto, para cancelar las tareas pendientes, destruir los *widgets* de los *plugins* y liberar las señales del proyecto anterior. A continuación llama a *open()* en *ProjectService*, que fija la carpeta y el nombre del proyecto y reconstruye su estado en el hilo de interfaz.

Si el proyecto tiene una señal, *ProjectService* la registra en el *DataStore* mediante *add_signal()* y la fija como señal activa con *set_active_signal()*. Si la frecuencia de muestreo había sido corregida a mano, la aplica con *set_sampling_rate()*. Después recorre los conjuntos de *trials* guardados: vuelve a recortar cada uno con sus parámetros, lo agrega a la señal con *add_trial_dataset()*, reaplica los descartes con *discard_trial()* y, si había recetas de preprocesamiento, registra el conjunto derivado con *register_derived_trials()*. Luego restaura las mediciones en el *DataStore* y, por cada extensión guardada, inicia el *plugin*, construye su *widget* y le entrega sus parámetros mediante *apply_analysis_params()*.

Finalmente, *ProjectService* establece $dirty = False$ y notifica a *MainWindow*, que actualiza el título de la ventana y el *Explorer* e informa al usuario con el mensaje **"*Project opened*"**.

Si la lectura falla (*failed*), *MainWindow* muestra un diálogo con el error; si el usuario la cancela desde la barra de progreso, *TaskService* confirma la cancelación (*cancelled*) y la interfaz muestra el mensaje **"*Open project canceled*"**. En ambos casos no se ejecuta *liberar_senal_anterior()*, por lo que el proyecto que estaba abierto se conserva intacto. Sea cual sea el resultado, *MainWindow* desmarca el proyecto como ocupado al terminar. De esta manera, el diagrama muestra que el estado anterior solo se reemplaza cuando la lectura del nuevo proyecto ha terminado correctamente.

4.2 Diagrama guardar proyecto

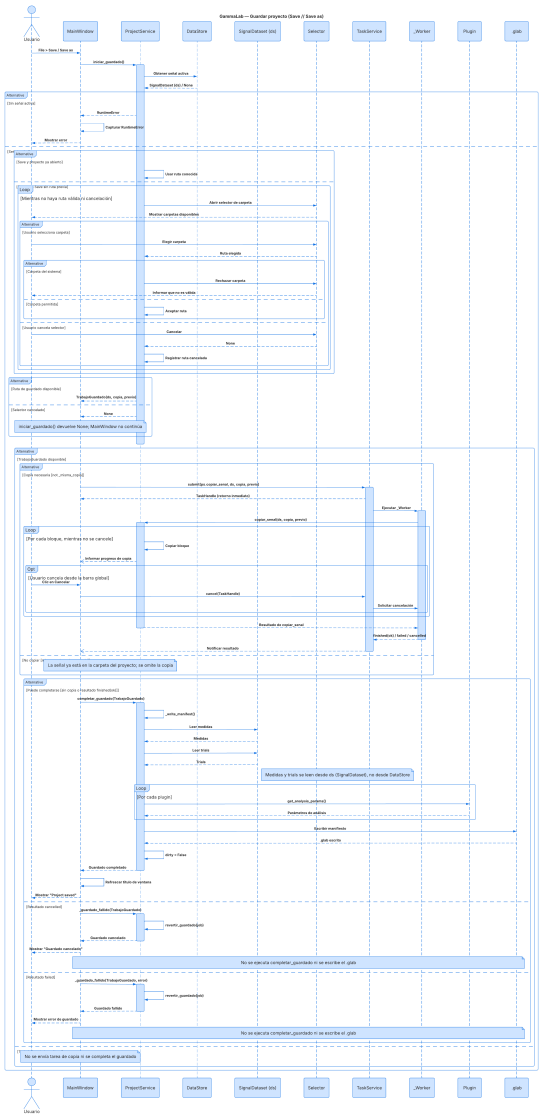


Figura 8: Diagrama de secuencia para guardar proyecto

Este diagrama representa el flujo que sigue GammaLab 2.0 al usar *save* y *save as* en un proyecto. El proceso comienza cuando el usuario solicita guardar el proyecto desde la interfaz principal *MainWindow*, que llama al método *iniciar_guardado()* de *ProjectService*. Este servicio verifica que exista una señal activa mediante *DataStore*. Si no hay una señal disponible, se genera un *RuntimeError*, que es capturado por *MainWindow* para informar al usuario y detener el proceso.

Cuando existe una señal cargada, *ProjectService* determina la ruta en la cual se guardará el proyecto. Si la ruta es válida, se utiliza directamente; de lo contrario, se abre el selector de

carpetas para que el usuario elija una ubicación. El sistema valida la carpeta seleccionada y, si no está permitida, informa al usuario y permite realizar una nueva selección. Este ciclo se mantiene hasta que la carpeta seleccionada sea válida o el usuario cancele la selección. En este último caso, *iniciar_guardado* devuelve *None* y *MainWindow* cancela la acción.

Una vez definida la ruta, *ProjectService* devuelve la información necesaria para realizar el guardado. Si la señal requiere ser copiada a la carpeta del proyecto, *MainWindow* envía la tarea a *TaskService*, que ejecuta el trabajo de forma asíncrona mediante *_Worker*. La copia se realiza por bloques y se informa el progreso al usuario. Durante esta etapa, el usuario puede cancelar el proceso desde la barra de progreso global; la solicitud se transmite a *TaskService* y al trabajador encargado de la operación. En caso de que la señal ya se encuentre en la carpeta del proyecto, se omite esta tarea de copia.

Cuando la copia finaliza correctamente, o cuando no era necesaria, *MainWindow* llama al método *completar_guardado()* dentro de *ProjectService*. Allí se genera el manifiesto del proyecto, se consultan las medidas y los *trials* directamente desde *SignalDataset*, y se recorren los *plugins* para obtener sus parámetros de análisis mediante *get_analysis_params()*. Finalmente, se escribe el archivo *.glab*, se establece *dirty* = *False* y se notifica a *MainWindow* que el guardado se completó. La interfaz actualiza el título de la ventana y muestra el mensaje de confirmación **"*Project saved*"**.

En caso contrario, si la copia es cancelada o falla, *MainWindow* ejecuta *_guardado_fallido()*, que revierte el estado temporal del guardado y comunica el resultado correspondiente al usuario. En estos casos no se ejecuta *completar_guardado()* ni se escribe el archivo *.glab*. De esta manera, el diagrama muestra que la generación del archivo del proyecto depende de que todas las operaciones previas finalicen correctamente.

4.3 Diagrama cerrar proyecto

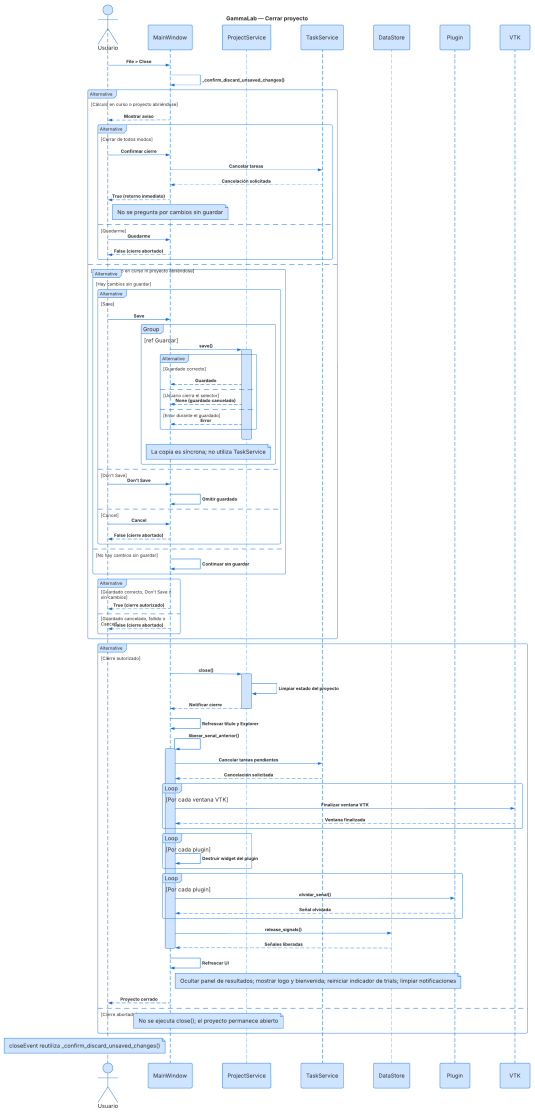


Figura 9: Diagrama de secuencia para cerrar proyecto

Este diagrama representa el flujo que sigue GammaLab 2.0 cuando el usuario solicita cerrar un proyecto desde la opción ***File > Close project***. El proceso comienza en *MainWindow*, que invoca *_confirm_discard_unsaved_changes()* para determinar si el cierre puede realizarse de forma segura y si es necesario solicitar una confirmación al usuario.

En primer lugar, se verifica si hay un cálculo en ejecución o si el proyecto se encuentra en proceso de apertura. Cuando se presenta alguna de estas condiciones, la interfaz muestra un aviso que permite al usuario decidir entre cerrar de todos modos o permanecer en el proyecto. Si el usuario confirma el cierre, *MainWindow* solicita a *TaskService* la cancelación

de las tareas y autoriza inmediatamente el cierre. En esta situación no se realiza una consulta adicional sobre los cambios sin guardar. Si el usuario decide quedarse, el método devuelve *False* y el cierre se interrumpe.

Cuando no hay cálculos en ejecución ni procesos de apertura, el sistema verifica si existen cambios sin guardar. Si los hay, presenta las opciones ***Save***, ***Don't Save*** y ***Cancel***. Al seleccionar ***Save***, *MainWindow* solicita a *ProjectService* ejecutar *save()*. Este guardado se realiza de manera síncrona y no utiliza *TaskService*. Si el guardado termina correctamente, el flujo puede continuar; si el usuario cancela el selector de guardado o se produce un error, el cierre se aborta. Al seleccionar ***Don't Save***, se omite el guardado y se permite continuar. En cambio, ***Cancel*** interrumpe el procedimiento y mantiene el proyecto abierto. Si no existen cambios pendientes, el sistema también puede continuar sin guardar.

Una vez autorizada la operación, *MainWindow* llama a *close()* en *ProjectService*, que limpia el estado del proyecto y notifica el cierre a la interfaz. Posteriormente, se actualizan el título de la ventana y el *Explorer*. A continuación, *MainWindow* ejecuta *liberar_senal_anterior()* para liberar los recursos asociados a la señal y al proyecto. Este procedimiento comienza solicitando la cancelación de las tareas pendientes mediante *TaskService*. Después, se recorren las ventanas de *VTK* para finalizarlas, se destruyen los *widgets* de los *plugins* y se llama a *olvidar_senal()* en cada *plugin* para desvincularlo de la señal anterior. Finalmente, *DataStore* ejecuta *release_signals()* para liberar las señales almacenadas.

Como última etapa, *MainWindow* actualiza la interfaz: oculta el panel de resultados, muestra el logo y la pantalla de bienvenida, reinicia el indicador de *trials* y limpia las notificaciones. Una vez terminadas estas operaciones, informa al usuario que el proyecto se ha cerrado.

Si en cualquiera de las validaciones iniciales el cierre es cancelado o no puede completarse, no se ejecuta *ProjectService.close()* y el proyecto permanece abierto. El diagrama también muestra que el evento *closeEvent* reutiliza *_confirm_discard_unsaved_changes()*, manteniendo el mismo mecanismo de confirmación cuando el usuario intenta cerrar la ventana principal.

4.4 Diagrama *wavelet*

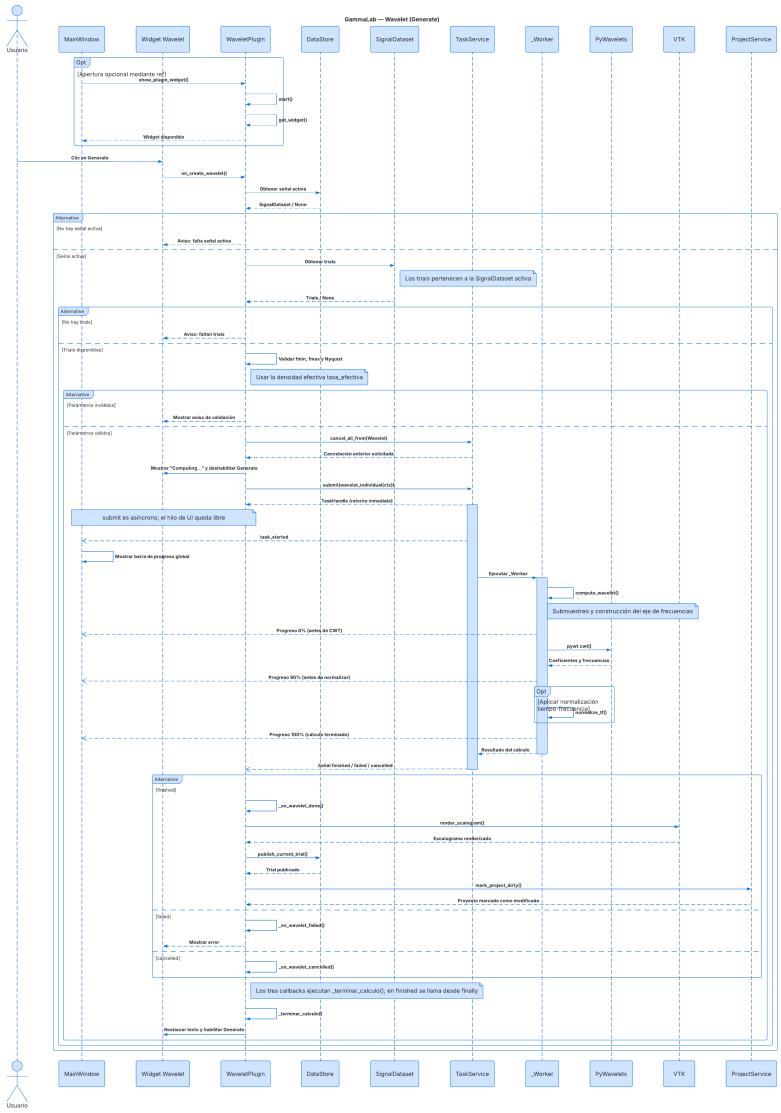


Figura 10: Diagrama de secuencia para *wavelet*

Este diagrama representa el flujo que sigue GammaLab 2.0 al generar un análisis con *Wavelet* a partir de los *trials* de la señal activa. Este flujo comienza con la apertura del *widget* de *wavelet* desde *MainWindow*. Para ello, se solicita al *WaveletPlugin* que inicialice el *plugin* y proporcione su interfaz mediante *get_widget()*. Una vez disponible el *widget*, el usuario puede seleccionar los parámetros del análisis y ejecutar la opción ***Generate***.

Al iniciar la generación, el *widget* invoca *on_create_wavelet()* en *WaveletPlugin*. El *plugin* consulta la señal activa y verifica que exista un objeto *SignalDataset*. Si no hay una señal activa, se muestra una advertencia y el proceso se detiene. Si existe, se obtienen sus

trials directamente desde *SignalDataset*, ya que estos pertenecen a la señal activa. Cuando no hay *trials* disponibles, también se informa al usuario y no se continúa con el cálculo.

Si existen *trials*, el *plugin* valida los parámetros de entrada, incluyendo los límites de frecuencia (*fmin* y *fmax*) y la frecuencia de Nyquist. Para esta validación se utiliza la densidad de muestreo efectiva (*tasa_efectiva*). Si los parámetros no son válidos, se muestra el aviso correspondiente. En caso contrario, el *plugin* solicita la cancelación de cálculos anteriores relacionados con *Wavelet* mediante *TaskService*, cambia el estado del *widget* a ***"Computing..."*** y deshabilita el botón ***Generate*** para evitar que se inicie otra ejecución mientras el cálculo está en curso.

A continuación, *WaveletPlugin* envía el cálculo a *TaskService* mediante *submit (wavelet_individual (ctx))*. Este servicio devuelve inmediatamente un *TaskHandle* y ejecuta el trabajo de forma asíncrona a través de *_Worker*, permitiendo que el hilo de la interfaz permanezca libre. El trabajador realiza el cálculo *Wavelet* mediante *compute_wavelet ()*, que utiliza PyWavelets (*pywt.cwt*) para obtener los coeficientes y las frecuencias correspondientes. De acuerdo con la configuración del análisis, también puede aplicarse una normalización en el dominio tiempo-frecuencia mediante *normalize_tf ()*.

Durante la ejecución se actualiza el progreso global: se informa un avance inicial antes de ejecutar la transformada, un avance aproximado del 90 % antes de la normalización y el 100 % cuando el cálculo termina. Al finalizar, *TaskService* comunica el resultado al *plugin* mediante una de las señales disponibles: *finished*, *failed* o *cancelled*.

Cuando el cálculo finaliza correctamente, se ejecuta *_on_wavelet_done()*. El *plugin* solicita a *VTK* la representación del escalograma mediante *render_scalogram()*, publica el *trial* generado a través de *publish_current_trial()* y notifica a *ProjectService* que el proyecto ha sido modificado mediante *mark_project_dirty()*. Si ocurre un error, *_on_wavelet_failed()* se encarga de mostrar el mensaje correspondiente; si el usuario cancela la operación, *_on_wavelet_cancelled()* gestiona ese resultado.

Finalmente, los tres *callbacks* ejecutan *_terminar_calculo()* para restaurar el estado del *widget*, recuperar el texto original del botón y habilitar nuevamente la opción ***Generate***. Así, el diagrama representa tanto el procesamiento exitoso como las rutas de validación, error y cancelación, mostrando la interacción entre la interfaz, el *plugin*, el sistema de tareas, las herramientas de procesamiento y la visualización.

5 INTERFAZ DE USUARIO

Esta sección presenta la interfaz de GammaLab 2.0 tal como la ve el usuario. Primero se describe la organización de la pantalla principal y sus elementos; después, las pestañas de la barra de navegación en el orden en que suelen usarse: *File*, para cargar señales y gestionar proyectos; *Preprocessing*, para dividir la señal en *trials* y prepararlos; y *Analysis*, con los análisis disponibles. Cada elemento se acompaña de una captura de la aplicación. Los componentes de código que implementan estos elementos se describen en la Sección 3.3.

5.1 Organización de la pantalla principal

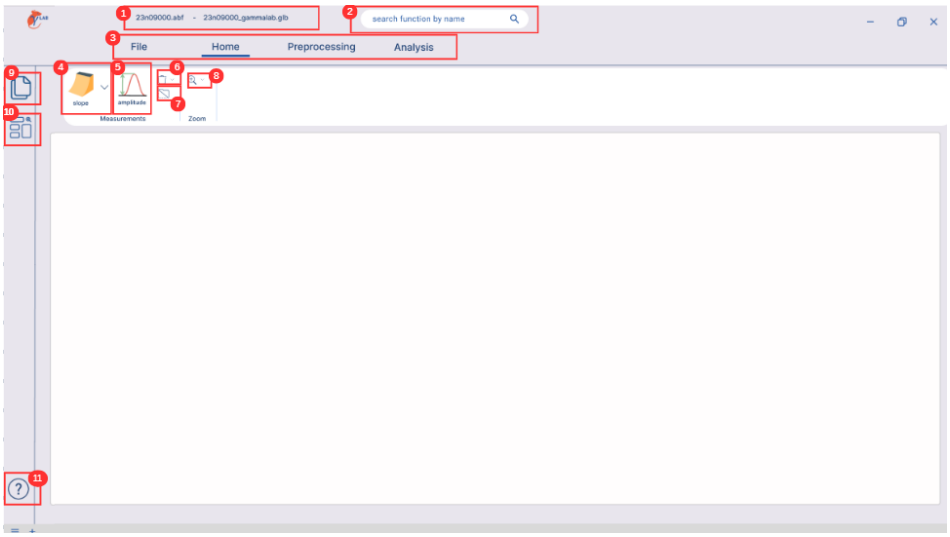


Figura 11: Organización de la pantalla principal de Gamma Lab

La pantalla principal se compone de los siguientes elementos, numerados en la Figura 11:

- **1 - Panel de control:** muestra el nombre de la señal abierta junto al nombre, editable, del proyecto activo, guardado con extensión *.glab*.
- **2 - Barra de búsqueda:** ubicada en la parte superior, permite localizar y acceder rápidamente a una funcionalidad específica del sistema sin tener que recorrer manualmente la barra de navegación.
- **3 - Barra de navegación:** agrupa las pestañas *File*, *Home*, *Preprocessing* y *Analysis*, permitiendo alternar entre la gestión de archivos, las mediciones sobre la señal, el preprocesamiento y los módulos de análisis.

- **4 - *slope*:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la pendiente de la señal entre dos puntos seleccionados por el usuario, mediante las subopciones *Slope 2 Trials* (calculada sobre dos ensayos) y *Slope All Trials* (calculada sobre la totalidad de los ensayos generados).

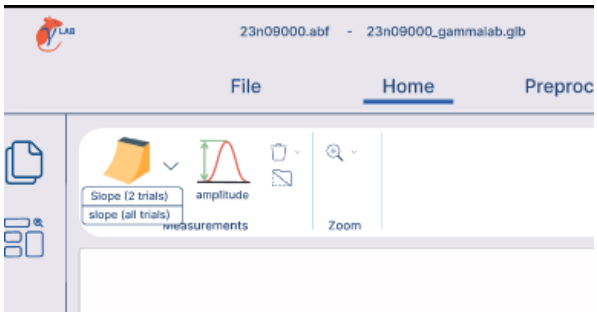


Figura 12: opciones de *slope*

- **5 - *amplitude*:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la amplitud de la señal entre dos puntos seleccionados por el usuario sobre la gráfica.
- **6 - eliminar medida:** permite eliminar la última medición de *slope* o *amplitude* realizada, o bien la totalidad de las mediciones registradas sobre la señal.



Figura 13: opciones de eliminar medida

- **7 - mostrar/ocultar:** permite mostrar u ocultar en pantalla las mediciones de *slope* y *amplitude* realizadas previamente sobre la señal, sin necesidad de eliminarlas.
- **8 - *zoom*:** activa o desactiva la función de acercamiento sobre la gráfica de la señal, permitiendo observar con mayor detalle un segmento de la señal y las mediciones realizadas sobre él.



Figura 14: opciones de *zoom*

- **9 - explorador de archivos:** despliega, desde la barra lateral izquierda, la carpeta del sistema de archivos en la que se encuentra almacenado el proyecto actual junto con las señales y resultados exportados, sin salir de la aplicación.

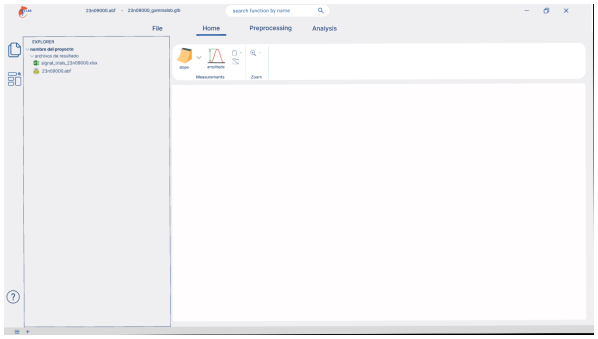


Figura 15: explorador de archivos desplegado

- **10 - resultados:** despliega, desde la barra lateral izquierda, una ventana inferior con los resultados numéricos de las mediciones de *slope* y *amplitude* realizadas sobre la señal, los cuales pueden exportarse en formato *.csv* o *.xlsx* para su análisis externo.

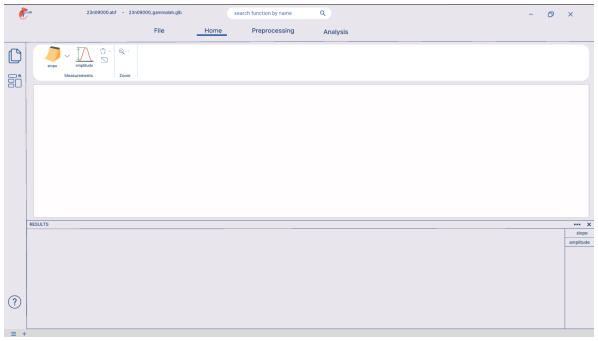


Figura 16: resultados desplegados

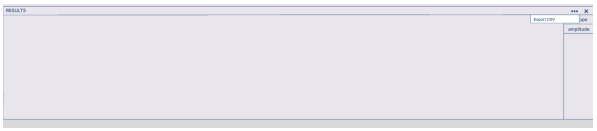


Figura 17: Exportación de los resultados en formato CSV o XLSX

- **11 - ayuda:** despliega, desde la barra lateral izquierda, el manual de usuario del sistema, con información de referencia sobre el uso de cada funcionalidad.

5.2 Carga y guardado de archivos (pestaña *File*)

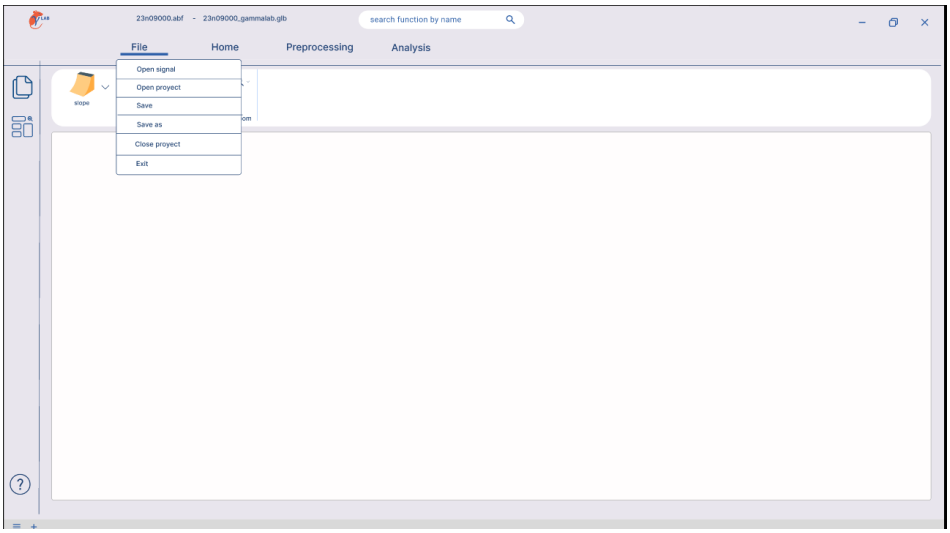


Figura 18: Pestaña *File* de Gamma Lab

La pestaña *File* permite gestionar la carga y el guardado de señales y proyectos. Agrupa las siguientes opciones:

- ***Open signal*:** permite cargar una señal individual dentro de la aplicación para su preprocesamiento y análisis.
- ***Open project*:** permite abrir un proyecto previamente guardado con extensión *.glab*, restaurando la señal activa, los *trials*, los análisis y los parámetros de la sesión.
- ***Save*:** guarda los cambios realizados sobre el proyecto actualmente abierto.
- ***Save as*:** permite guardar el proyecto actual bajo un nuevo nombre o ubicación, sin sobrescribir el archivo original.

- ***Close project*:** cierra el proyecto que se encuentra abierto, liberando la señal activa de memoria.
- ***Exit*:** cierra la aplicación, con la misma confirmación de cambios sin guardar que *Close project*.

A continuación se ilustra el flujo de carga de una señal y de un proyecto:

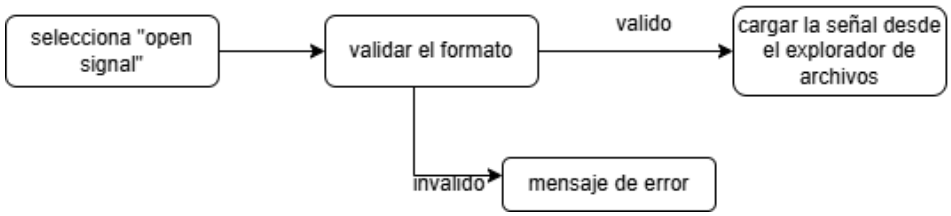


Figura 19: Flujo de carga de la señal (*Open signal*)

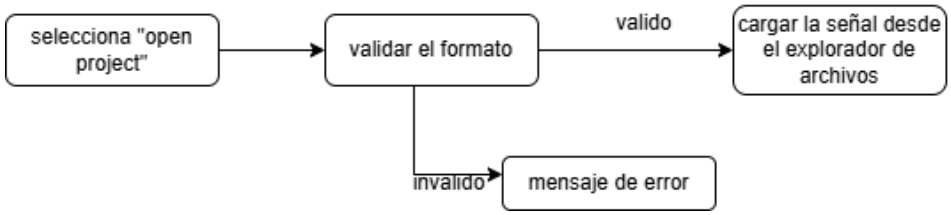


Figura 20: Flujo de carga de proyecto (*Open project*)

Mediante las opciones *Save* y *Save as* es posible almacenar el estado del espacio de trabajo en un proyecto: una carpeta que contiene el manifiesto con extensión *.glab*, una copia de la señal y la subcarpeta de exportaciones. El manifiesto registra la señal activa, los parámetros con que se generaron los *trials*, sus descartes y el preprocesamiento aplicado, las mediciones realizadas y los parámetros de los análisis; los resultados numéricos no se guardan, sino que se recalculan al abrir el proyecto (Sección 3.1.4).

5.3 Preprocesamiento

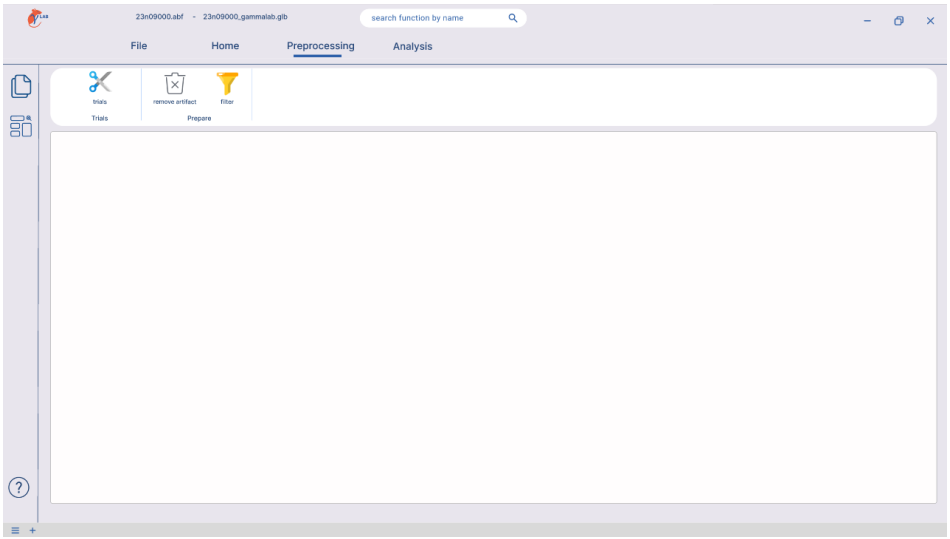


Figura 21: Pestaña *Preprocessing* de Gamma Lab

Categoría *Trials*

- ***trials*:** divide la señal cargada en ensayos (*trials*), permitiendo seleccionar el canal a dividir, el canal de estímulos, el umbral (*threshold*), el número de estímulos y el tamaño de la ventana de segmentación. Una vez generados, el usuario puede navegar entre los *trials* resultantes e incluirlos o descartarlos individualmente del conjunto que participará en los análisis posteriores.

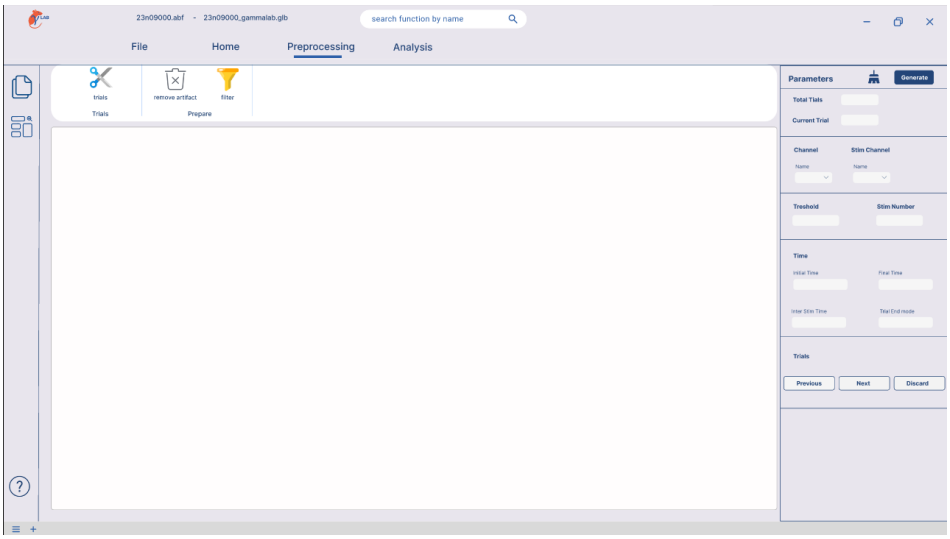


Figura 22: opciones de *trials*

Categoría *prepare*

- ***remove artifact*:** elimina artefactos presentes en los *trials* mediante la selección de un rango de tiempo a corregir, ofreciendo dos modos de corrección: interpolación del segmento seleccionado o eliminación de la señal desde el inicio del *trial* hasta el punto seleccionado.

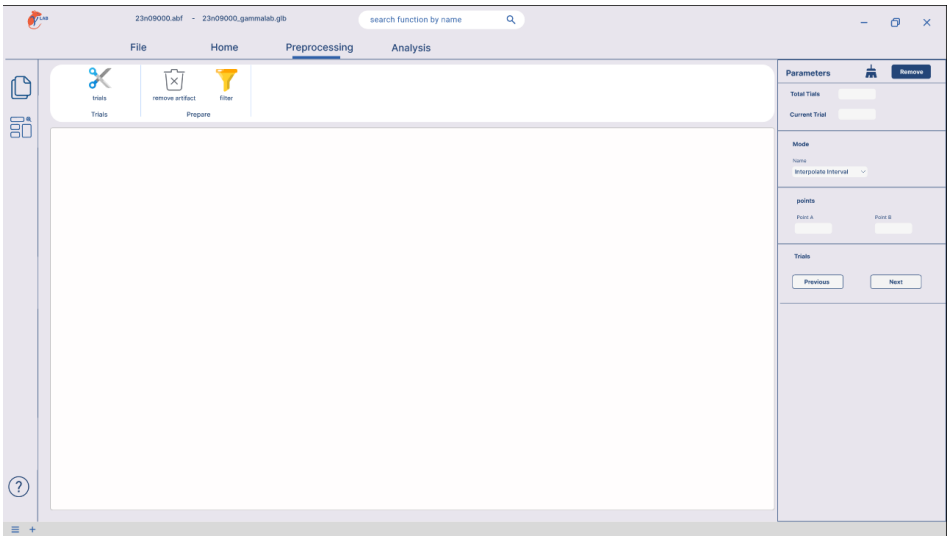


Figura 23: opciones de *remove artifact*

- ***filter*:** aplica un filtro digital Butterworth a la señal, mediante la selección del rango de frecuencias de corte y el orden del filtro, mostrando el resultado tanto sobre la señal completa como sobre el primer *trial* generado.

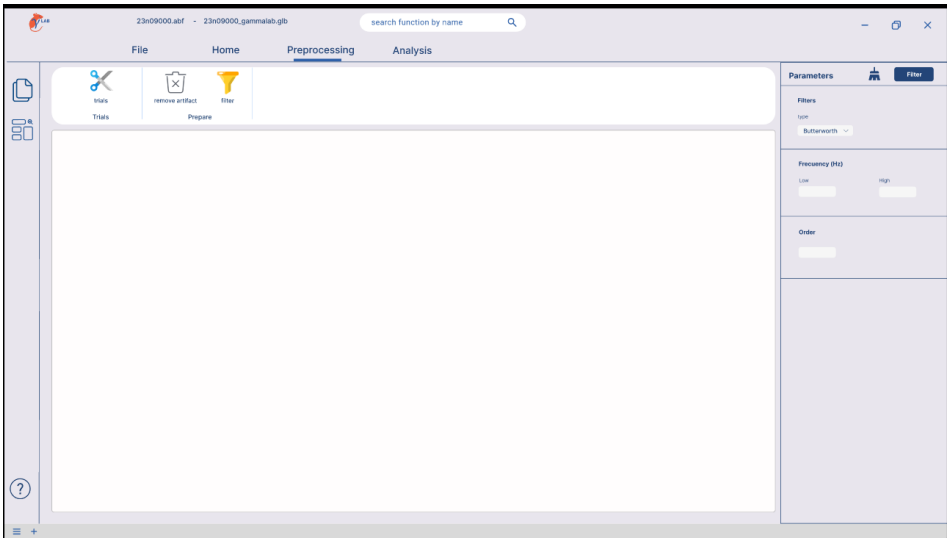


Figura 24: opciones de *filter*

5.4 *Analysis*

En esta sección se pueden realizar diferentes análisis de la señal, a continuación se describen los análisis disponibles y sus opciones:

- **FFT:** calcula la transformada rápida de Fourier de la señal, mediante la configuración de la frecuencia de muestreo y el rango de frecuencias de interés, y muestra el espectro de potencia resultante de forma individual para cada ensayo.

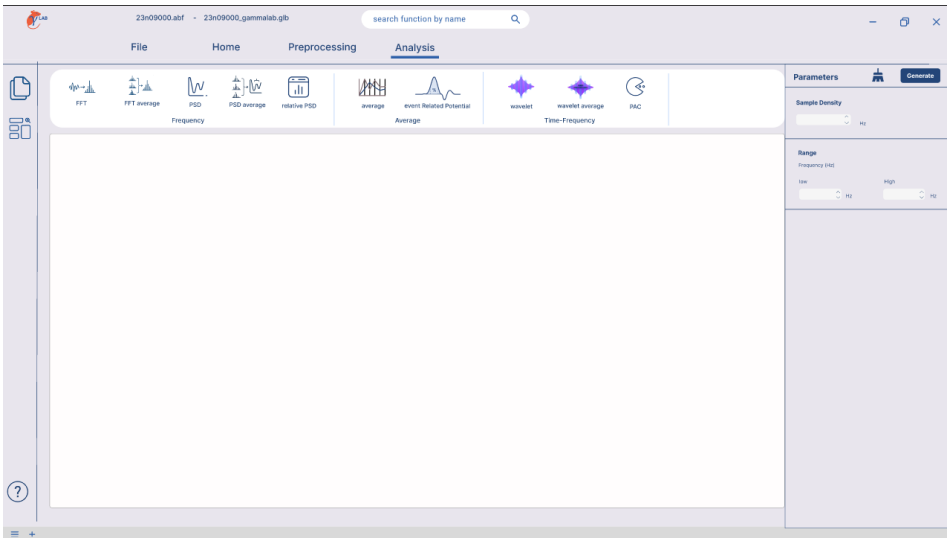


Figura 25: opciones de FFT

- **FFT *Average*:** calcula la FFT de la señal con la misma configuración de frecuencia de muestreo y rango de frecuencias que la opción anterior, pero adicionalmente promedia los espectros obtenidos de todos los ensayos para reducir el ruido y resaltar los componentes de frecuencia comunes entre ellos.

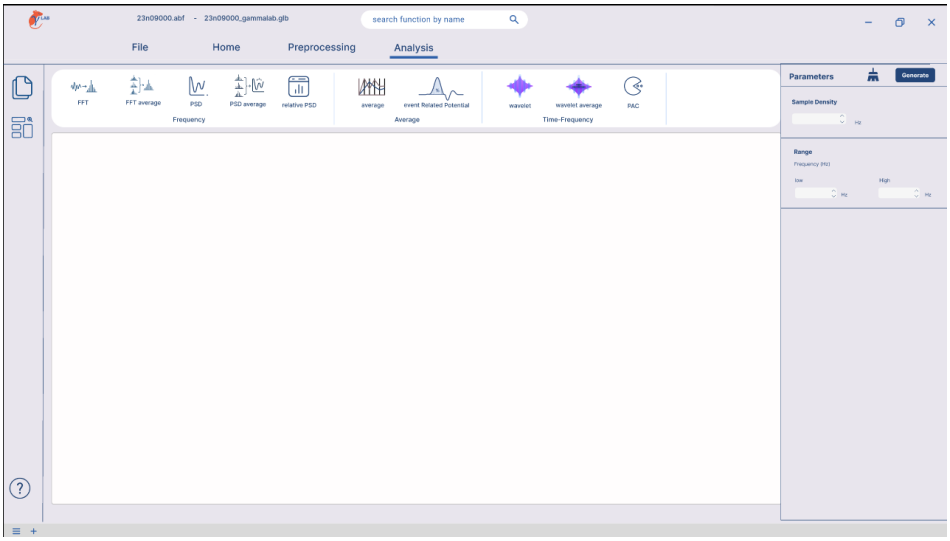


Figura 26: opciones de FFT *Average*

- **PSD:** estima la densidad espectral de potencia de la señal mediante el método de Welch, permitiendo configurar el modo de cálculo, el tipo de ventana, el rango de frecuencias y el modo de *detrend*, y muestra el resultado de forma individual para cada ensayo.

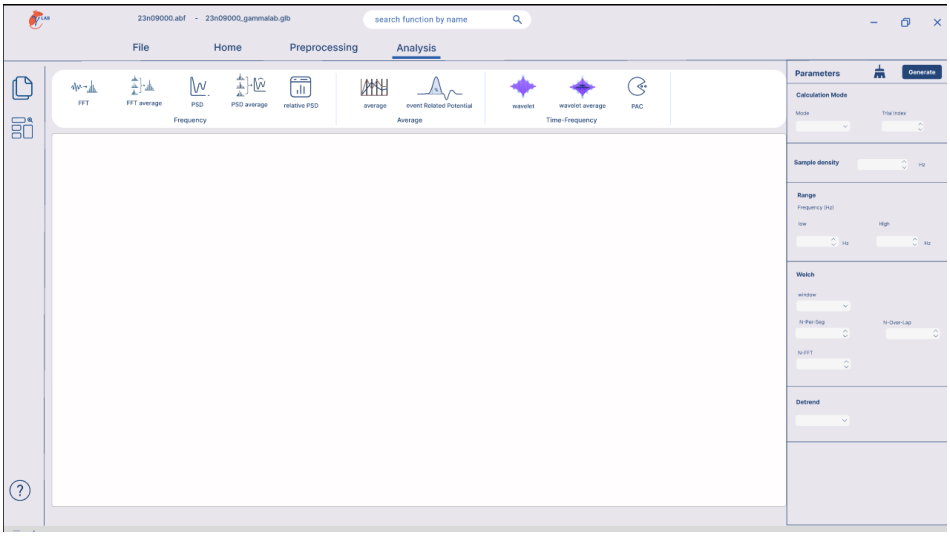


Figura 27: opciones de PSD

- ***PSD Average*:** estima la PSD de la señal mediante el método de Welch, con la misma configuración de frecuencia de muestreo, ventana y rango de frecuencias que la opción anterior, pero promediando de forma agregada los resultados de todos los ensayos.

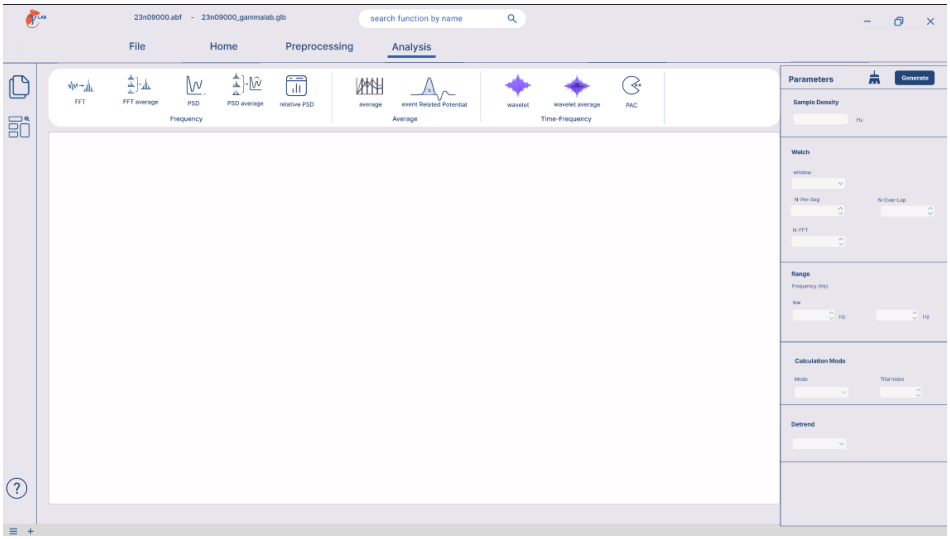


Figura 28: opciones de *PSD Average*

- ***Relative PSD*:** calcula la PSD relativa de la señal, entregando tanto la potencia absoluta como la potencia relativa aportada por cada banda de frecuencia del rango seleccionado, mediante la configuración del tipo de ventana, el modo de *detrend* y el modo de cálculo.
- ***Average*:** calcula y visualiza el promedio temporal de los *trials* generados, útil para observar patrones comunes y atenuar variaciones aleatorias entre ensayos.

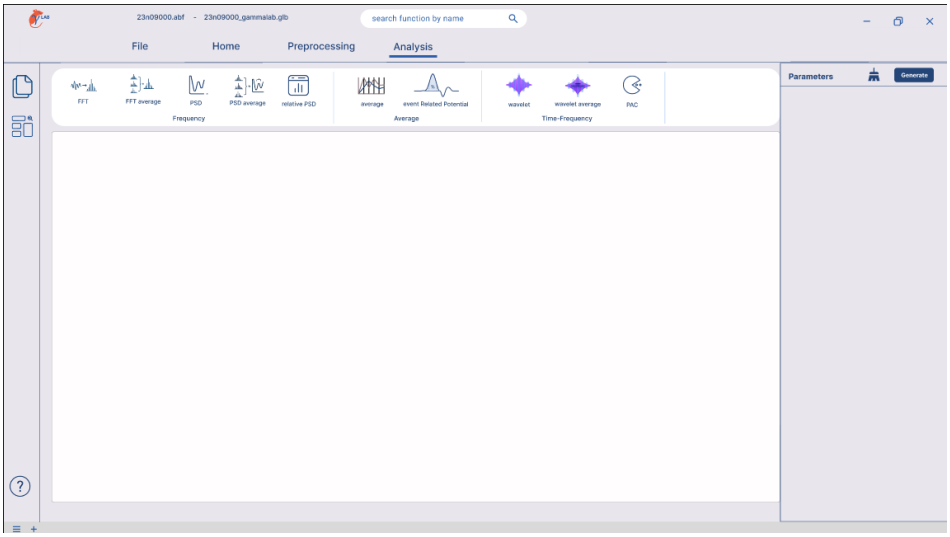


Figura 29: opciones de *Average*

- ***Event Related Potential*:** calcula el potencial relacionado con eventos (ERP), generando un *Butterfly Plot* con todos los *trials* alineados y un espectrograma de la dinámica tiempo-frecuencia asociada al evento.

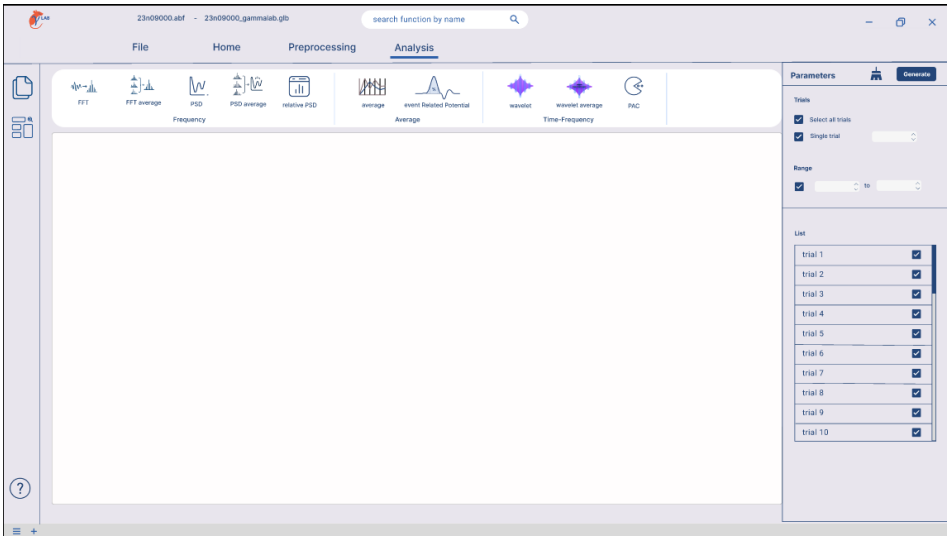


Figura 30: opciones de *Event Related Potential*

- ***Wavelet*:** calcula la transformada *wavelet* continua de la señal, mediante la configuración de la frecuencia de muestreo, el rango de frecuencias, el número de ciclos, la normalización y la escala de visualización, permitiendo observar cómo varía el contenido espectral de la señal a lo largo del tiempo para cada ensayo.

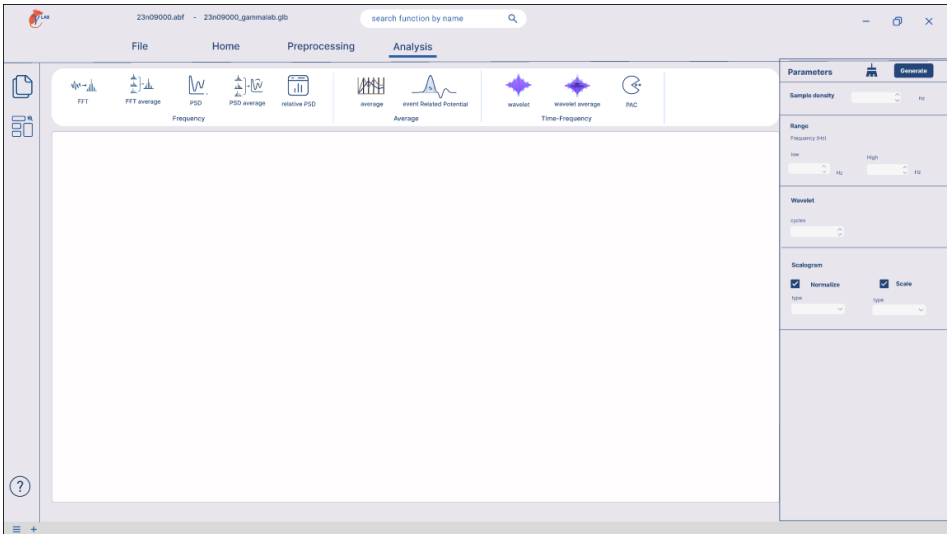


Figura 31: opciones de *Wavelet*

- ***Wavelet Average*:** calcula la transformada *wavelet* con la misma configuración que la opción anterior, y promedia los resultados sobre múltiples *trials*, generando un mapa tiempo-frecuencia agrupado.

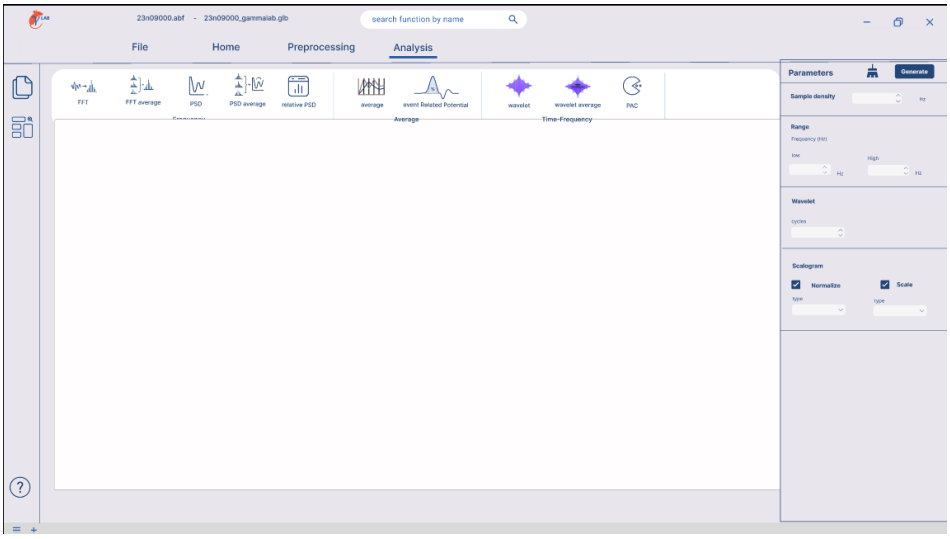


Figura 32: opciones de *Wavelet Average*

- **PAC:** calcula el acoplamiento de fase-amplitud (*Phase-Amplitude Coupling*), evaluando la relación entre la fase de una banda de baja frecuencia y la amplitud de un rango de frecuencias seleccionado, mediante cuatro modos: *PAC*, sobre un *trial* individual; *PAC Average*, sobre el promedio de todos los *trials*; *PAC Psel*, que concatena los *trials*, detecta los ciclos completos del ritmo lento con una *wavelet* de Gabor y promedia ventanas centradas en sus valles; y *PAC Psel inter-canal*, que toma la fase de un canal y la amplitud de otro. El resultado incluye el mapa de acoplamiento fase-frecuencia, el histograma de distribución de amplitud en función de la fase, y el vector de acoplamiento medio con su magnitud y ángulo.

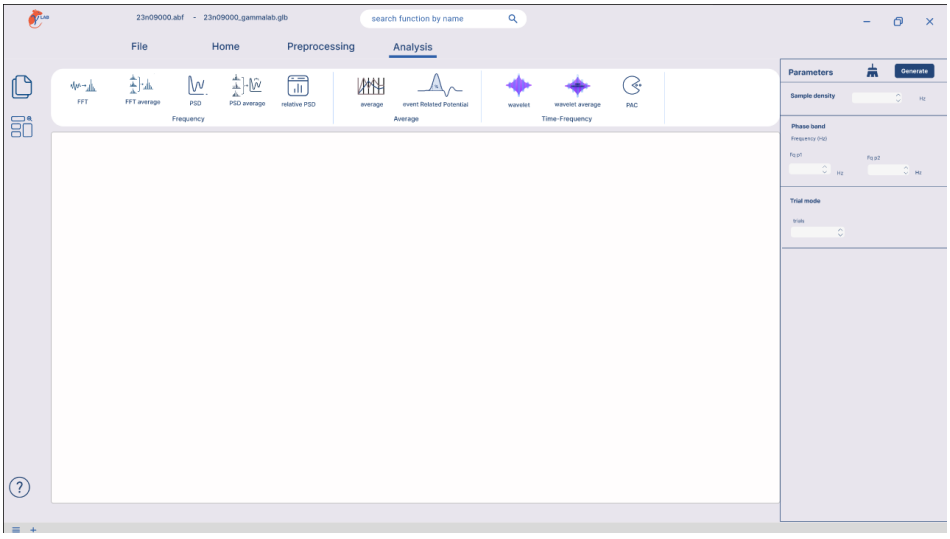


Figura 33: opciones de PAC

- ***Modulation Index*:** calcula el índice de modulación del acoplamiento fase-amplitud sobre una rejilla de bandas, definida por el rango de frecuencias de fase (*Fq P1* a *Fq P2*, con su paso) y el de amplitud (*Fq A1* a *Fq A2*, con su paso). Para cada par de bandas filtra la señal, obtiene la fase y la envolvente mediante la transformada de Hilbert y calcula el índice de Canolty y su variante normalizada, que se presentan como un comodulograma. A diferencia de PAC, no utiliza la transformada *wavelet*.

- **PPC:** mide la consistencia de la diferencia de fase entre dos canales a lo largo del tiempo, mediante la consistencia de fase por pares (*Pairwise Phase Consistency*) de Vinck et al. El usuario selecciona los dos canales, el rango de frecuencias y el número de ciclos de la *wavelet* de Gabor, y el resultado es un mapa de frecuencia contra tiempo en el que se dejan en blanco las celdas que no pueden medirse con las muestras disponibles.

6 TRAZABILIDAD Y GESTIÓN DEL DISEÑO

Este capítulo relaciona los requisitos y los casos de uso que el sistema cubre con la manera en que se cubrieron y con las pruebas que lo verifican. Las fuentes son el SRS para los requisitos, el SAD para los casos de uso, el código fuente y la hoja de seguimiento del proyecto. El estado corresponde al 1 de noviembre de 2026.

De los 81 requisitos del SRS, 81 están cubiertos. De los 23 casos de uso del SAD, 23 están cubiertos; un caso se considera cubierto cuando lo están todos los requisitos que lo componen. La verificación se apoya en 761 pruebas automáticas. En la ejecución del 8 de noviembre de 2026 se aprobaron 755; 5 comparaciones con las referencias de MATLAB quedaron como fallo esperado, con su causa documentada, y 1 prueba falló por estar desactualizada frente a la incorporación de PPC. El detalle se presenta en el TD. Donde un límite de tiempo o de memoria no se presta a una prueba automática, la verificación es una medición, con su cifra. Los nombres de componentes, extensiones y pruebas aparecen entre comillas y en cursiva.

6.1 Requisitos cubiertos

La Tabla 22 recoge los 45 requisitos funcionales cubiertos y la Tabla 23 los 36 no funcionales. Cada fila indica cómo se cubrió el requisito y cómo se verificó.

Requisito	Cómo se cubrió	Verificación
FR-IO-01 R08	**Persistir la señal cargada.** "*DataStore*" conserva la señal como señal activa de la sesión y "*FileIOService*" la lee sin duplicarla en memoria.	Pruebas automáticas del almacén de datos y de la lectura sin duplicación ("*test_dataset*", "*test_fileio_sin_duplicar*").
FR-IO-02 R80	**Cargar señales ABF, EDF y MAT.** "*FileIOService*" tiene un lector para cada formato y la extensión "*Open Signal*" elige el lector según el archivo.	Pruebas automáticas de la lectura de ABF y EDF; la de MAT se comprueba en la prueba del programa instalado ("*test_fileio_abf*", "*test_fileio_edf*", "*prueba_congelado*").
FR-IO-03 R07	**Guardar un proyecto.** "*ProjectService*" escribe el archivo del proyecto con el nombre y la ruta de la señal y el trabajo realizado, sin los datos de la señal, que se copia por bloques como archivo aparte junto al proyecto en un hilo de trabajo; "*MainWindow*" coordina el guardado.	Pruebas automáticas del guardado en pasos, de su cancelación y del registro de la ruta ("*test_guardado_proyecto*", "*test_correcciones_interaccion*").

Requisito	Cómo se cubrió	Verificación
FR-IO-04 R78	**Abrir un proyecto guardado.** "*ProjectService*" lee el proyecto, corrige la frecuencia y vuelve a cortar los *trials* fuera del hilo de la interfaz; después, en el hilo de la interfaz, registra la señal y restaura las mediciones y los análisis guardados. Si el archivo de la señal no está ni junto al proyecto ni en su ruta original, "*MainWindow*" avisa con su nombre.	Pruebas automáticas de la apertura, de la señal que falta y de la ruta de origen, y de la persistencia de las modificaciones de artefactos ("*test_apertura_proyecto*", "*test_persistencia_artifact_remove*", "*test_correcciones_interaccion*").
FR-IO-05 R05	**Exportar los resultados de los análisis.** "*ExportService*" escribe los resultados en CSV, Excel y JSON y las gráficas como imagen; el escritor de Excel es propio y no depende de bibliotecas externas.	Pruebas automáticas de la exportación y del escritor de Excel. "*Relative PSD*" dibuja su gráfica con el menú contextual y exporta parámetros, potencias y fracción por banda ("*test_export*", "*test_xlsx_writer*", "*test_resultados_xlsx*", "*test_relative_psd_exportacion*").
FR-IO-06 R09	**Una sola columna X en el CSV.** "*ExportService*" escribe una sola columna del eje X cuando todas las series lo comparten y una por serie cuando difieren.	Pruebas automáticas de la exportación con ejes iguales y distintos ("*test_export*").
FR-IO-07 R28	**Nombre de exportación descriptivo.** "*ExportService*" compone el nombre propuesto con el de la señal, el del canal y el de la extensión que genera la gráfica.	Verificación manual del nombre propuesto al exportar.
FR-UI-01 R01	**Cortar la señal en *trials*.** La extensión "*Trials*" detecta los estímulos en el canal elegido y corta la señal con la ventana indicada.	Pruebas automáticas de la detección de estímulos y del corte ("*test_trials*").
FR-UI-02 R24	**Modificar un *trial* ya creado.** "*Trials*" permite regenerar el conjunto con otros parámetros y descartar *trials*, y "*Remove Artifact*" y "*Filter*" aplican su resultado a los *trials* activos como conjuntos derivados, sin alterar el original.	Pruebas automáticas de los conjuntos derivados, que dejan el original intacto ("*test_trials_derivados*").
FR-UI-03 R25	**Restablecer parámetros por defecto.** Cada extensión con botón de ejecución tiene un botón "*Clear*" que devuelve sus controles a los valores por defecto sin tocar los datos; en "*Trials*" incluye los selectores de canal, canal de estímulo y modo, y "*Open Signal*" tiene el suyo.	Prueba automática del Clear de *Trials* y verificación manual en las demás extensiones ("*test_correcciones_interaccion*").

Requisito	Cómo se cubrió	Verificación
FR-UI-04 R26	**Valores X/Y bajo el cursor.** "*CursorReadout*" convierte la posición del puntero en valores de los ejes y, cuando la gráfica dibuja curvas, muestra los del punto de la señal más cercano al cursor.	Pruebas automáticas de la conversión de píxeles a datos, del formato de la lectura y del punto más cercano ("*test_indicadores_ui*", "*test_correcciones_interaccion*").
FR-UI-05 R27	**Elegir los *trials* de cada análisis.** "*Average*", "*Event Related Potential*", "*FFT Average*", "*PSD Average*", "*Wavelet Average*" y "*Relative PSD*" permiten marcar los *trials* que participan; los análisis de un solo *trial*, incluida "*Wavelet*", permiten elegir cuál, y "*SignalDataset*" conserva dentro del conjunto los *trials* descartados.	Pruebas automáticas de la selección y de los descartes ("*test_average_seleccion*", "*test_indicadores_ui*", "*test_correcciones_interaccion*").
FR-UI-06 R29	**Ajustar la frecuencia de muestreo.** "*SignalDataset*" recalcula el eje de tiempo al cambiar la frecuencia, invalida sus cachés y puede restaurar el valor del archivo; "*Open Signal*" ofrece el control.	Pruebas automáticas del cambio y de la restauración de la frecuencia ("*test_cu018_sampling_rate*").
FR-UI-07 R30	***Zoom* en todas las direcciones.** "*VTKContextMenu*" y la cinta "*Home*" ofrecen *zoom* con la rueda del ratón (con Ctrl en el eje X y con Shift en el eje Y) y con botones para acercar, alejar y restablecer.	Pruebas automáticas de los ejes del *zoom* y verificación manual en cada gráfica ("*test_correcciones_interaccion*").
FR-UI-08 R31	**Mostrar el umbral en *mV*.** "*Trials*" muestra el umbral con la unidad *mV* y lo convierte a la unidad del canal de estímulo antes de detectar; si ese canal no es de voltaje, muestra su propia unidad.	Pruebas automáticas de la conversión de unidades y verificación manual del campo ("*test_trials*").
FR-UI-09 R32	**Mostrar "*Butterfly*" explícitamente.** La extensión "*Event Related Potential*" titula la gráfica como "*Butterfly Plot*" seguido del canal y de los *trials* usados.	Prueba automática del título dibujado ("*test_correcciones_interaccion*").
FR-UI-10 R36	**Promediar todos los *trials*.** "*Average*" promedia muestra a muestra todos los *trials* del canal activo.	Pruebas automáticas del promedio ("*test_average_plugin*").
FR-UI-11 R37	**Indicar el *trial* actual.** Cada extensión publica el *trial* en que trabaja y "*MainWindow*" lo muestra en un indicador permanente.	Pruebas automáticas del estado y del texto del indicador ("*test_indicadores_ui*").

Requisito	Cómo se cubrió	Verificación
FR-UI-12 R38	**Indicar qué *trials* participan.** Cada operación deja constancia de los *trials* usados (por ejemplo, "*trials* 2 a 4 de 6") en el título del resultado y en el proyecto.	Pruebas automáticas de la descripción de los *trials* y de la selección de "*Average*" ("*test_indicadores_ui*", "*test_average_seleccion*").
FR-UI-13 R40	**Remover un artefacto con medidas.** "*Remove Artifact*" calcula y muestra las medidas del segmento y pide confirmación antes de aplicar la modificación.	Pruebas automáticas del cálculo de las medidas y del uso del servicio de tareas ("*test_medidas_artefacto*", "*test_artifact_remove_orquestador*").
FR-UI-14 R41	**Promediar los *trials* seleccionados.** "*Average*" promedia solo los *trials* elegidos (uno, un rango o una selección manual) y rotula el resultado con ellos.	Pruebas automáticas de la selección y del promedio ("*test_average_seleccion*").
FR-UI-15 R68	***Trials* identificados por archivo y canal.** "*SignalDataset*" identifica cada conjunto de *trials* por archivo y canal y devuelve el del canal pedido, sin sustituirlo por el último generado.	Pruebas automáticas de la consulta de *trials* por canal ("*test_trials_por_canal*").
FR-WV-01 R33	**Escala y rango de color del escalograma.** "*Wavelet*" y "*Wavelet Average*" ofrecen escala lineal o logarítmica y un rango de color automático o fijado a mano.	Pruebas automáticas del rango de color ("*test_escalograma_rango_color*").
FR-WV-02 R34	**Normalizar el escalograma.** "*Wavelet*" y "*Wavelet Average*" normalizan el escalograma con el método que elige el usuario.	Pruebas automáticas de las dos extensiones, con las fallas de comparación numérica descritas en DT-03 ("*test_wavelet_plugin*", "*test_wavelet_average_plugin*", "*test_submuestreo_wavelet*").
FR-WV-03 R35	**Exportar el escalograma en EMF.** "*ExportService*" escribe el gráfico en formato vectorial EMF desde el menú contextual.	Pruebas automáticas del archivo EMF ("*test_emf_export*").
FR-PAC-01 R13	**Ejecutar el análisis PAC.** "*PAC*" ejecuta el cálculo en el servicio de tareas y dibuja el resultado en la misma pantalla.	Pruebas automáticas del cálculo de PAC ("*test_pac_core*").
FR-PAC-02 R14	**Banda de la fase.** "*PAC*" pide la banda de la fase y filtra la señal en ella antes de calcular.	Pruebas automáticas del cálculo de PAC ("*test_pac_core*").
FR-PAC-03 R15	**Banda de la amplitud.** "*PAC*" pide el rango de frecuencias de la amplitud y calcula la transformada en ese rango.	Pruebas automáticas del cálculo de PAC ("*test_pac_core*").
FR-PAC-04 R16	**Intervalo temporal o *trials* del análisis.** "*PAC*" calcula sobre un *trial*, sobre el promedio de los *trials* o sobre todos concatenados, según el modo elegido.	Pruebas automáticas del selector de *trial* y verificación manual de los modos ("*test_pac_intercanal_plugin*").

Requisito	Cómo se cubrió	Verificación
FR-PAC-05 R17	**Calcular el acoplamiento por fase y frecuencia.** El cálculo obtiene el mapa con la transformada *wavelet* de Morse y agrupa la amplitud por intervalos de fase, como la referencia de MATLAB.	Pruebas automáticas contra la referencia de MATLAB (“*test_pac_core*”).
FR-PAC-06 R18	**Mapa de acoplamiento fase-frecuencia.** “*PAC*” dibuja el mapa con la fase en el eje X, en grados de 0 a 360, y la frecuencia de la amplitud en el eje Y.	Pruebas automáticas del eje de fase y verificación manual del mapa (“*test_pac_grados*”).
FR-PAC-07 R19	**Distribución de amplitud según la fase.** “*PAC*” dibuja el histograma de la amplitud media por intervalo de fase.	Pruebas automáticas del cálculo del histograma y del eje de fase (“*test_pac_core*”, “*test_pac_grados*”).
FR-PAC-08 R20	**Vector de acoplamiento medio.** El cálculo obtiene el vector medio a partir del histograma y “*PAC*” muestra su magnitud y su ángulo.	Pruebas automáticas del cálculo del vector (“*test_pac_core*”).
FR-PAC-09 R21	**Exportar los números de PAC.** “*PAC*” entrega a “*ExportService*” los parámetros, el resumen, el histograma y el mapa de la última ejecución, para CSV, Excel y JSON.	Pruebas automáticas de la exportación y de los ángulos en grados (“*test_export*”, “*test_pac_grados*”).
FR-PAC-10 R22	**Exportar las gráficas de PAC.** El menú contextual de “*PAC*” exporta las gráficas como imagen o en formato vectorial.	Pruebas automáticas de la exportación de imágenes y de EMF (“*test_export*”, “*test_emf_export*”).
FR-PAC-11 R23	**Guardar los parámetros de cada ejecución.** “*PAC*” conserva los parámetros y el resultado de la última ejecución y los entrega a “*ProjectService*” para guardarlos en el proyecto.	Prueba automática de la restauración de análisis al abrir un proyecto (“*test_apertura_proyecto*”).
FR-PAC-12 R60	**Elegir el canal de fase y el de amplitud.** “*PAC*” ofrece el modo entre canales con un selector del canal de amplitud; el canal de fase es el activo.	Pruebas automáticas del selector y de su comportamiento (“*test_pac_intercanal_plugin*”).
FR-PAC-13 R61	**Mapa de acoplamiento entre canales.** El cálculo recibe los dos canales y toma los ciclos y la fase del de fase y la amplitud del otro; con el mismo canal en los dos da el resultado de un canal.	Pruebas automáticas del cálculo entre canales (“*test_pac_intercanal*”).
FR-PAC-14 R62	**Mismas visualizaciones que PAC de un canal.** El modo entre canales usa la misma representación que el PAC de un solo canal.	Pruebas automáticas de que ambos modos dibujan los mismos paneles (“*test_pac_intercanal_plugin*”).

Requisito	Cómo se cubrió	Verificación
FR-PAC-15 R63	**PPC: dos canales y rango de frecuencias.** "*PPC*" permite elegir dos canales y el rango de frecuencias y calcula la coherencia de fase con un estimador corregido.	Pruebas automáticas del cálculo de "*PPC*" y de su ejecución en el servicio de tareas ("*test_ppc_core*", "*test_ppc_plugin_orquestado*").
FR-PAC-16 R64	**PPC: mapa de frecuencia y tiempo.** "*PPC*" dibuja la coherencia como un mapa de frecuencia contra tiempo, con el eje de frecuencia logarítmico.	Pruebas automáticas del cálculo y de la transformada de Gabor compartida ("*test_ppc_core*", "*test_gabor_en_core*").
FR-PAC-17 R65	**Exportar los números de PPC y del PAC entre canales.** "*PPC*" y el PAC entre canales exportan sus resultados en CSV, Excel y JSON; en el segundo, los parámetros incluyen el canal de amplitud.	Pruebas automáticas que exportan archivos reales de PAC entre canales ("*test_pac_intercanal_exportacion*").
FR-PAC-18 R66	**Exportar las gráficas de PPC y del PAC entre canales.** "*PPC*" y el PAC entre canales usan el mismo menú contextual y el mismo "*ExportService*" que el PAC de un canal, para imagen y formato vectorial.	Pruebas automáticas de la exportación de imágenes y de EMF; no hay prueba específica del modo entre canales ("*test_emf_export*").
FR-PAC-19 R75	**PAC Psel segmenta por ciclo de la onda lenta.** "*PAC*" en modo Psel detecta los ciclos de la onda lenta con la transformada de Gabor, toma cada ciclo como época y las superpone en una gráfica de tipo butterfly.	Pruebas automáticas de la selección de ciclos y de la transformada de Gabor ("*test_pac_core*", "*test_gabor_en_core*").
FR-PAC-20 R76	**Filtro Butterworth pasa-banda de fase cero.** "*Filter*" aplica un filtro Butterworth pasa-banda de fase cero con las frecuencias de corte y el orden que elige el usuario, y es el único tipo que ofrece; la vista previa usa el canal de los *trials*.	Pruebas automáticas de los filtros y medición de su tiempo ("*test_pac_core*", "*test_preprocessing_bench*", "*test_correcciones_interaccion*").

Tabla 22: Requisitos funcionales cubiertos

Requisito	Cómo se cubrió	Verificación
NFR-U-01 R02	**Diferenciar información, aviso y error.** "*PluginAlerts*" ofrece tres niveles de severidad y cada situación usa el que le corresponde; la información y los avisos no interrumpen al usuario.	Pruebas automáticas de los niveles de mensaje ("*test_niveles_de_mensaje*", "*test_indicadores_ui*").

Requisito	Cómo se cubrió	Verificación
NFR-U-02 R03	**Resultado en la misma pantalla.** Cada extensión dibuja su resultado en el área de gráficos de la pantalla en que se pidió.	Verificación manual en cada extensión.
NFR-U-03 R04	**Retroalimentación en operaciones de más de 1 s.** "*TaskProgressBar*" muestra el avance y el botón de cancelar de toda tarea del servicio de tareas; el guardado y la apertura de proyectos también informan su progreso.	Pruebas automáticas del guardado y de la apertura. Medición: abrir un proyecto bloquea la interfaz unos 80 ms y guardar un archivo de 314 MB, 18 ms ("*test_task_service*", "*test_guardado_proyecto*", "*test_apertura_proyecto*").
NFR-U-04 R06	**Iconos en las funcionalidades.** Cada extensión declara su icono y "*MainWindow*" lo muestra en el botón de la cinta.	Verificación manual de la cinta.
NFR-U-05 R10	**Bienvenida a usuarios nuevos.** "*MainWindow*" muestra la primera vez que se ejecuta la aplicación una ventana que explica las funciones básicas, y en "*Home*" una pantalla de bienvenida mientras no hay un módulo abierto.	Prueba automática de que la ventana sale solo la primera vez; verificación manual de la pantalla ("*test_correcciones_interaccion*").
NFR-U-06 R12	**Comunicar el cierre del proyecto.** Al cerrar un proyecto, o al cargar otra señal con un proyecto abierto, "*MainWindow*" pregunta si se descartan los cambios, cierra el proyecto, comunica su cierre en la barra de estado y restablece el título y la pantalla de bienvenida.	Pruebas automáticas del cierre al cargar otra señal y verificación en la aplicación ("*test_correcciones_interaccion*").
NFR-U-07 R42	**Descripción emergente de cada control.** Cada extensión declara su descripción para su botón de la cinta, y un servicio completa la descripción de los controles de cada pantalla (campos, selectores y botones) con una propia o con la de su rótulo.	Prueba automática que recorre la interfaz de cada extensión y falla si algún control queda sin descripción ("*test_ayudas_controles*").
NFR-U-08 R43	**Funciones de edición en *Home*.** La cinta "*Home*" reúne las mediciones, el borrado, la visibilidad de las mediciones y el *zoom*.	Verificación manual de la pantalla.
NFR-U-09 R44	**Pantalla de inicio durante la carga.** "*SplashScreen*" muestra el avance de la carga de los servicios y de las extensiones durante el arranque.	Verificación manual del arranque.

Requisito	Cómo se cubrió	Verificación
NFR-P-01 R11	**Liberar la señal al cerrar el proyecto.** "*MainWindow*" cancela las tareas, destruye los *widgets* de las extensiones y pide a cada una soltar sus datos; "*DataStore*" quita las señales, los *trials*, las mediciones y el *trial* actual.	Pruebas automáticas y medición: tras cerrar el proyecto no queda ninguna copia viva de la señal ("*test_liberar_senal*").
NFR-P-02 R45	**Liberar recursos gráficos al salir del módulo.** Al salir de una extensión, "*MainWindow*" cierra su ventana gráfica después de mostrar la siguiente; seis extensiones de análisis lo hacen y diez conservan su ventana para no superar el tiempo de regreso permitido.	Pruebas automáticas de la liberación. Medición: al recorrer los módulos de análisis, el proceso pasa de 3.118 MB sin liberar a 2.585 MB con la configuración actual ("*test_liberar_ventanas*").
NFR-P-03 R47	**Cargar el archivo en 2 s o menos.** "*Open Signal*" lee el archivo en el servicio de tareas con "*FileIOService*" y dibuja la señal al terminar.	Pruebas automáticas de la lectura sin duplicación. Medición con un archivo de 2 canales de 1,8 millones de muestras: 0,04 s de lectura y 0,2 s hasta verlo en pantalla (límite: 2 s) ("*test_io_bench*", "*test_fileio_sin_duplicar*").
NFR-P-04 R48	***Wavelet* en 10 s o menos.** El cálculo de "*Wavelet*" se hace por tandas de frecuencias, lo que acota el tiempo y la memoria.	Medición: de 1,1 a 1,4 s para un *trial* de 30.500 muestras con los parámetros por defecto (límite: 10 s) ("*test_compute_bench*").
NFR-P-05 R49	**Graficar 3 canales sin degradar la interacción.** Las gráficas muestran como máximo 2000 puntos por serie, que se recalculan al hacer *zoom*, de modo que la interacción se mantiene fluida con varios canales.	Pruebas automáticas del submuestreo. Medición en el equipo de desarrollo, con tres canales de 1,8 millones de muestras: mediana de 29 a 44 FPS al hacer *zoom* y de 24 a 36 FPS al desplazar (límite: 24 FPS); el peor cuadro de cada corrida varió entre 16 y 31 FPS según la carga del equipo. Reducir los puntos dibujados hasta 250 no modificó la tasa, por lo que el tiempo restante es fijo del renderizado ("*test_submuestreo*", "*test_render_bench*").
NFR-P-06 R50	**Cambiar de módulo en 500 ms o menos.** "*MainWindow*" cambia de extensión conservando la señal activa en memoria, sin volver a leerla del disco.	Medición: el cambio de módulo tarda entre 10 y 310 ms con una señal de 1,7 millones de muestras (límite: 500 ms).
NFR-P-07 R51	**Exportar 100 000 filas en 2 s o menos.** El escritor de Excel propio genera directamente el contenido de cada hoja, sin pasar por una biblioteca externa.	Pruebas automáticas del escritor. Medición con 100.000 filas: 0,39 s en Excel y 0,11 s en CSV (límite: 2 s) ("*test_xlsx_writer*").

Requisito	Cómo se cubrió	Verificación
NFR-P-08 R52	**Liberar la señal anterior en menos de 1 s.** Al confirmarse la carga, "*Open Signal*" pide a "*MainWindow*" liberar la señal anterior y a "*DataStore*" quitarla.	Pruebas automáticas y medición: la señal anterior se libera en 243 a 353 ms; tras cuatro cargas seguidas queda una sola señal viva y la memoria sube 178 MB en vez de 631 MB ("*test_liberar_senal*").
NFR-P-09 R53	**Cierre en menos de 3 s con recursos liberados.** Al cerrar, "*MainWindow*" cancela las tareas, finaliza las ventanas gráficas y suelta los *widgets*.	Medición: la ventana se cierra en 172 ms y el proceso termina en 0,63 s (límite: 3 s).
NFR-P-10 R55	**Interfaz responsiva (0,3 s o menos).** El cómputo corre fuera del hilo de la interfaz y "*PAC*" difiere la creación de su ventana gráfica para no congelar la interfaz.	Medición: la interfaz se congela entre 270 y 480 ms la primera vez que se abre PAC, según la corrida, y entre 110 y 200 ms en adelante (límite: 300 ms).
NFR-P-11 R56	**Promedio de todos los *trials* en 1 s o menos.** El promedio se calcula en una sola operación vectorizada sobre la matriz de *trials*.	Medición: de 0,32 a 0,41 s para 50 *trials* de 30.500 muestras (límite: 1 s) ("*test_compute_bench*").
NFR-P-12 R57	**Exportar una gráfica en 1 s (3 s si es vectorial).** "*ExportService*" captura la ventana gráfica para las imágenes y escribe el EMF directamente.	Pruebas automáticas de la exportación. Medición: PNG en 0,26 s, TIFF en 0,06 s y EMF en 0,12 s (límites: 1 s y 3 s) ("*test_export*", "*test_emf_export*").
NFR-P-13 R59	**Conversión en bloque a las estructuras de VTK.** Los adaptadores convierten matrices completas en tablas de VTK de una sola vez, sin recorrerlas elemento por elemento.	Medición: 1,8 millones de filas convertidas en 0,03 s. Prueba automática del llenado vectorizado del mapa de calor ("*test_vtk_adapters_bench*", "*test_erp_vtk_llenado*").
NFR-P-14 R71	**Cómputo pesado por el servicio de tareas.** "*IPlugin*" ofrece un único patrón para enviar un cómputo al servicio de tareas, y las extensiones lo usan para la carga, el corte, el filtrado, los artefactos, los análisis y los promedios.	Pruebas automáticas del patrón y de las extensiones que lo usan ("*test_run_task*", "*test_task_service*", "*test_artifact_remove_orquestador*", "*test_ppc_plugin_orquestado*").
NFR-P-15 R73	**Hasta 2000 puntos por serie y 200 curvas.** El submuestreo conserva el mínimo y el máximo de cada tramo y limita las curvas dibujadas a 200, también en el butterfly del ERP; los datos siguen completos para exportar y medir.	Pruebas automáticas del submuestreo: como máximo 2000 puntos por serie, sin alterar los datos ("*test_submuestreo*", "*test_correcciones_interaccion*").
NFR-P-16 R74	**PAC por el servicio de tareas.** "*PAC*" envía el comodulograma al servicio de tareas, que informa el progreso y permite cancelarlo.	Medición: un comodulograma de 20×20 pares tarda entre 0,2 y 0,5 s (límite: 30 s); la cancelación se comprueba en la aplicación.

Requisito	Cómo se cubrió	Verificación
NFR-P-17 R81	**Listo en 8 s o menos.** El arranque instancia los servicios y descubre las extensiones mientras muestra la pantalla de inicio.	Medición: 3,1 s hasta la ventana lista (límite: 8 s) y prueba del programa instalado (“*test_kernel_bench*”, “*prueba_congelado*”).
NFR-R-01 R46	**Terminar los hilos al cancelar o cerrar.** “*TaskService*” cancela de forma cooperativa y, si la tarea no responde en dos segundos, la desliga, descarta su resultado y libera la interfaz y la cola; “*MainWindow*” cancela las tareas de una extensión al salir de ella. La redacción del requisito se ajustó a este comportamiento: la terminación forzada del hilo queda fuera de su alcance.	Pruebas automáticas de la cancelación cooperativa, del desligado a los dos segundos y del cambio de sección (“*test_task_service*”, “*test_cancelacion_al_cambiar_seccion*”).
NFR-R-02 R54	**Sincronizar el estado compartido.** “*DataStore*” admite un solo escritor, el hilo de la interfaz, rechaza las escrituras de otro hilo y protege las lecturas con un candado.	Pruebas automáticas de la concurrencia sobre el almacén (“*test_data_store_concurrencia*”).
NFR-R-03 R58	**Retirar los detectores de eventos de VTK.** Al salir de una extensión, “*MainWindow*” retira los detectores de eventos de sus gráficas y los repone al volver; cuando una extensión recrea el menú contextual de una gráfica, el menú nuevo retira los detectores y el menú del anterior.	Pruebas automáticas del retiro, la reposición y el reemplazo del menú; verificación en la aplicación con cinco cálculos seguidos (“*test_vtk_observers*”).
NFR-R-04 R72	**Reutilizar la representación gráfica.** “*SignalDataset*” reutiliza la tabla de VTK de la señal mientras sus datos no cambian y la descarta al cambiar la frecuencia de muestreo; los *trials* y los resultados se vuelven a dibujar cada vez.	Pruebas automáticas que verifican que la segunda petición devuelve la misma tabla, que corregir o restaurar la frecuencia la descarta y que la tabla nueva trae el eje corregido sin tocar la anterior (“*test_cache_tabla_vtk*”, “*test_cu018_sampling_rate*”).
NFR-R-05 R79	**No modificar la señal ni los *trials* originales.** El preprocesamiento registra su resultado en “*SignalDataset*” como un conjunto derivado y conserva el original, que puede restaurarse.	Pruebas automáticas de los conjuntos derivados (“*test_trials_derivados*”).
NFR-S-01 R39	**Ejecutable instalable que registra la extensión del proyecto.** Se congela la aplicación, se prueba y se empaqueta un instalador para todo el equipo que registra la extensión “*.glab*”; al abrir un proyecto con doble clic, el programa recibe la ruta y lo abre.	Pruebas automáticas de las rutas del programa instalado y prueba del ejecutable, además de la instalación real verificada en un equipo (“*test_instalacion_rutas*”, “*prueba_congelado*”).

Requisito	Cómo se cubrió	Verificación
NFR-S-02 R67	**Vía pública única sobre los *trials*.** "*SignalDataset*" ofrece la única vía para leer, modificar e invalidar los *trials* y sus descartes; ninguna extensión alcanza su estructura interna.	Pruebas automáticas de los *trials* originales, derivados y por canal ("*test_trials_derivados*", "*test_trials_por_canal*").
NFR-S-03 R69	**Servicios compartidos registrados en el núcleo.** Los servicios de ajustes, exportación y medición se crean una sola vez en el registro del núcleo y los gráficos y las extensiones los piden al "*Kernel*". "*ExportService*" entrega a cada ventana un exportador y "*MeasurementService*" una sesión de medición por gráfico, y ambos guardan en un único lugar lo que comparten.	Prueba automática que recorre el código y falla si otro módulo crea uno de los servicios, y pruebas de cada servicio. Verificado en la aplicación con una exportación y una medición reales ("*test_servicios_compartidos*").
NFR-S-04 R70	**Cómputo pesado por un servicio del núcleo.** "*TaskService*" está registrado en el núcleo y gestiona la cola, la cancelación y el progreso; las extensiones lo usan a través de "*IPlugin*".	Pruebas automáticas del servicio de tareas y del patrón de uso ("*test_task_service*", "*test_run_task*").
NFR-S-05 R77	**Nuevo *plugin* sin modificar el núcleo.** El descriptor de cada extensión declara si guarda su análisis con el proyecto y el orden de sus botones; el núcleo lee esos datos en lugar de listas escritas a mano.	Pruebas automáticas con una extensión de prueba que se guarda, se restaura y se ordena sin tocar el núcleo ("*test_plugin_sin_tocar_nucleo*").

Tabla 23: Requisitos no funcionales cubiertos

6.2 Casos de uso cubiertos

La Tabla 24 recoge los 23 casos de uso cubiertos, con los componentes que participan y la secuencia principal. Se verifican con las pruebas de los requisitos que los componen (Tablas 22 y 23); la última columna indica los requisitos y las pruebas principales. La apertura, el guardado y el cierre de un proyecto y la transformada *wavelet* tienen su diagrama de secuencia en el capítulo 4.

Caso de uso	Componentes	Cómo se cubre	Verificación
CU-001 PAC de un canal	"*PAC*", "*TaskService*", "*VTKContextMenu*", "*ExportService*"	1) El botón de la cinta muestra la extensión. 2) Al crear, la extensión lee los parámetros y los *trials* activos. 3) El servicio de tareas ejecuta el cálculo en un hilo de trabajo. 4) El resultado llega al hilo de la interfaz, que dibuja los gráficos y actualiza el menú contextual.	Requisitos R02, R13 a R23, R75. Pruebas principales: "*test_pac_core*", "*test_pac_grados*", "*test_export*".
CU-002 PAC de dos canales y PPC	"*PAC*", "*PPC*", "*TaskService*"	1) La extensión pide los *trials* de cada canal y comprueba que estén alineados. 2) Encola el cálculo entre canales (PAC) o el de coherencia (PPC). 3) PPC reparte los *trials* entre cuatro hilos. 4) Se dibuja el mapa y se habilita la exportación.	Requisitos R60 a R66. Pruebas principales: "*test_pac_intercanal_plugin*", "*test_ppc_core*", "*test_pac_intercanal*".
CU-003 Cargar una señal con gestión de memoria	"*Open Signal*", "*TaskService*", "*FileIOService*", "*MainWindow*", "*DataStore*"	1) La lectura corre en el servicio de tareas. 2) Al terminar, la extensión pide a la ventana principal liberar la señal anterior. 3) Se registra la señal nueva y se deja activa. 4) Un evento avisa a la interfaz.	Requisitos R02, R07, R08, R11, R12, R47, R52, R54, R71, R80. Pruebas principales: "*test_correcciones_interaccion*", "*test_fileio_sin_duplicar*", "*test_liberar_senal*".
CU-004 Generar y modificar *trials*	"*Trials*", "*SignalDataset*", "*TaskService*"	1) La extensión valida el umbral y el canal de estímulo. 2) El corte corre en el servicio de tareas. 3) Al terminar guarda la receta con la que se generaron los *trials* y los registra en la señal. 4) Emite el evento de *trials* generados y muestra el primero.	Requisitos R01, R02, R24, R27, R31, R67, R71, R79. Pruebas principales: "*test_trials_derivados*", "*test_trials*", "*test_indicadores_ui*".
CU-005 Guardado y exportación de datos	"*ProjectService*", "*MainWindow*", "*ExportService*"	El guardado se describe en la Sección 4.2. Exportación: 1) El menú contextual pide a la exportación escribir las tablas o la gráfica en el formato elegido. 2) Se confirma al usuario.	Requisitos R02, R04, R05, R07, R09, R28, R51, R57. Pruebas principales: "*test_export*", "*test_guardado_proyecto*", "*test_xlsx_writer*".

Caso de uso	Componentes	Cómo se cubre	Verificación
CU-006 Seleccionar *trials* para el análisis	"*SignalDataset*", "*Average*", "*MainWindow*"	1) El usuario descarta o elige *trials*. 2) La extensión obtiene los índices válidos y arma la etiqueta de los *trials* usados. 3) El análisis usa solo esa selección. 4) La extensión publica el *trial* actual y la ventana actualiza el indicador.	Requisitos R27, R37, R38, R68. Pruebas principales: "*test_indicadores_ui*", "*test_average_seleccion*", "*test_correcciones_interaccion*".
CU-007 Remover un artefacto con medidas	"*Remove Artifact*", "*SignalDataset*", "*TaskService*"	1) El usuario marca el segmento. 2) Se calculan y se muestran las medidas. 3) Tras confirmar, el cálculo corre en el servicio de tareas. 4) El resultado se registra como conjunto derivado y se avisa a los demás módulos.	Requisitos R02, R40, R67, R69, R71, R79. Pruebas principales: "*test_artifact_remove_orquestador*", "*test_trials_derivados*", "*test_niveles_de_mensaje*".
CU-008 Navegar y cambiar de módulo	"*MainWindow*", "*TaskService*"	1) El botón de la cinta pide mostrar la extensión. 2) La ventana principal cancela las tareas de la extensión anterior, la detiene, retira sus detectores de eventos y la oculta. 3) Si la extensión lo permite, cierra su ventana gráfica después de mostrar la nueva. 4) Repone los detectores de la nueva y la muestra.	Requisitos R06, R43, R45, R50, R55, R58. Pruebas principales: "*test_liberar_ventanas*", "*test_vtk_observers*".
CU-009 Limpiar un formulario	"*IPlugin*" y cada extensión	1) El usuario pulsa "*Clear*". 2) La extensión restablece sus controles a los valores por defecto, sin tocar los datos ni los resultados.	Requisitos R25. Pruebas principales: "*test_correcciones_interaccion*".
CU-010 Consultar la ayuda contextual	"*PluginMeta*", "*MainWindow*", "*Help*"	1) El texto emergente sale de la descripción que cada extensión declara en su descriptor. 2) Qt lo muestra al detener el puntero. 3) El manual completo se abre en la extensión de ayuda con el visor de PDF.	Requisitos R42. Pruebas principales: "*test_ayudas_controles*".

Caso de uso	Componentes	Cómo se cubre	Verificación
CU-011 Iniciar la aplicación	"*Kernel*", "*PluginManager*", "*SplashScreen*", "*MainWindow*"	Sección 2.4.3. 1) Guarda del temporizador de VTK. 2) Aplicación gráfica y pantalla de inicio. 3) Servicios de datos, lectura y tareas. 4) Descubrimiento y registro de las extensiones. 5) Ventana principal. 6) Si se recibió un proyecto como argumento, se abre.	Requisitos R10, R39, R44, R69, R70, R77, R81. Pruebas principales: "*test_correcciones_interaccion*", "*test_instalacion_rutas*", "*test_servicios_compartidos*".
CU-012 Interactuar visualmente con la señal	"*VTKContextMenu*", "*CursorReadout*", "*MeasurementService*"	1) Los eventos del ratón llegan por detectores que se pueden retirar. 2) La lectura bajo el cursor convierte píxeles en datos y muestra X e Y. 3) La rueda ajusta los ejes. 4) Al cambiar el eje inferior se vuelven a reducir los puntos visibles.	Requisitos R26, R30, R32, R49, R59. Pruebas principales: "*test_correcciones_interaccion*", "*test_indicadores_ui*", "*test_submuestreo*".
CU-013 Calcular la transformada *wavelet*	"*Wavelet*", "*TaskService*"	Sección 4.4. 1) La extensión toma el *trial* activo. 2) Encola el cálculo por tandas de frecuencias. 3) Dibuja el escalograma. 4) El rango de color puede fijarse a mano.	Requisitos R02, R33, R34, R46, R48, R70. Pruebas principales: "*test_task_service*", "*test_niveles_de_mensaje*", "*test_indicadores_ui*".
CU-014 Cancelar un procesamiento pesado	"*TaskProgressBar*", "*TaskService*", "*TaskHandle*", "*TaskContext*"	1) El usuario pulsa Cancelar en la barra de progreso. 2) El servicio de tareas activa el evento de cancelación. 3) El cálculo lo consulta y termina. 4) Si no responde en dos segundos, se desliga y se descarta su resultado.	Requisitos R46, R55, R70, R74. Pruebas principales: "*test_task_service*", "*test_cancelacion_al_cambiar_seccion*", "*test_run_task*".
CU-015 Promedio de *trials*	"*Average*", "*TaskService*"	1) La extensión resuelve los *trials* según el modo (todos, uno, rango o manual). 2) El promedio corre en el servicio de tareas. 3) Se dibuja con submuestreo.	Requisitos R03, R36, R41, R56, R71. Pruebas principales: "*test_average_plugin*", "*test_average_seleccion*", "*test_compute_bench*".

Caso de uso	Componentes	Cómo se cubre	Verificación
CU-016 Aplicar filtros digitales	"*Filter*", "*SignalDataset*", "*TaskService*"	1) El usuario elige la banda. 2) La vista previa filtra un *trial*. 3) Al aplicar, el filtrado de todos los *trials* corre en el servicio de tareas. 4) El resultado se registra como conjunto derivado; el original queda intacto.	Requisitos R58, R59, R71, R76, R79. Pruebas principales: "*test_vtk_observers*", "*test_vtk_adapters_bench*", "*test_erp_vtk_llenado*".
CU-017 Exportación gráfica vectorial (EMF)	"*VTKContextMenu*", "*ExportService*"	1) El menú contextual ofrece exportar como imagen vectorial. 2) La exportación recorre las series del gráfico activo. 3) Un escritor propio genera los registros EMF. 4) Se confirma al usuario.	Requisitos R28, R35, R57. Pruebas principales: "*test_emf_export*", "*test_export*".
CU-018 Ajustar la frecuencia de muestreo	"*Open Signal*", "*SignalDataset*"	1) El usuario indica la frecuencia. 2) La señal recalcula su eje de tiempo e invalida sus cachés. 3) Puede restaurarse el valor del archivo.	Requisitos R29, R67, R72. Pruebas principales: "*test_cu018_sampling_rate*", "*test_trials_derivados*", "*test_trials_por_canal*".
CU-019 Optimizar el renderizado (submuestreo)	Módulo de submuestreo y adaptadores a VTK	1) La extensión entrega la serie completa. 2) El submuestreo conserva el mínimo y el máximo de cada tramo (2000 puntos). 3) Con el *zoom* se recalcula. 4) La exportación y las mediciones leen la serie completa.	Requisitos R30, R49, R59, R73. Pruebas principales: "*test_correcciones_interaccion*", "*test_submuestreo*", "*test_render_bench*".
CU-020 Orquestador del núcleo	"*IPlugin*", "*TaskService*", "*TaskHandle*", "*TaskContext*", "*TaskProgressBar*"	1) La extensión envía el cálculo; el servicio descarta las pendientes del mismo dueño y encola. 2) Un hilo ejecuta una tarea a la vez. 3) El avance, el resultado, el error y la cancelación llegan al hilo de la interfaz. 4) La extensión comprueba que sea la tarea vigente antes de dibujar.	Requisitos R04, R46, R54, R55, R69 a R71, R77. Pruebas principales: "*test_task_service*", "*test_run_task*", "*test_guardado_proyecto*".
CU-021 Cerrar un proyecto	"*MainWindow*", "*ProjectService*", "*DataStore*"	Sección 4.3. 1) Se pregunta si se descartan los cambios. 2) El servicio de proyectos olvida la ruta. 3) La ventana principal cancela las tareas, destruye los *widgets*, pide a cada extensión soltar sus datos y vacía el almacén de datos.	Requisitos R02, R07, R11, R12, R52. Pruebas principales: "*test_correcciones_interaccion*", "*test_liberar_senal*", "*test_niveles_de_mensaje*".

Caso de uso	Componentes	Cómo se cubre	Verificación
CU-022 Cerrar la aplicación	"*MainWindow*", "*TaskService*"	1) Se pregunta si se descartan los cambios. 2) Se cancelan las tareas de todas las extensiones. 3) Se finalizan las ventanas gráficas. 4) Se ocultan y se sueltan los *widgets*.	Requisitos R45, R46, R53, R58. Pruebas principales: "*test_liberar_ventanas*", "*test_task_service*", "*test_cancelacion_al_cambiar_seccion*".
CU-023 Abrir un proyecto	"*ProjectService*", "*TaskService*", "*MainWindow*", "*DataStore*"	Sección 4.1. 1) El servicio de proyectos lee el proyecto y la señal fuera del hilo de la interfaz. 2) Se libera la señal anterior. 3) Se registra la señal y se restauran los *trials*, las mediciones y los análisis guardados.	Requisitos R02, R07, R08, R39, R47, R54, R71, R78. Pruebas principales: "*test_correcciones_interaccion*", "*test_fileio_sin_duplicar*", "*test_niveles_de_mensaje*".

Tabla 24: Casos de uso cubiertos